



myMove: IoT Enabled Smart Parking Reservation and
Privacy Vehicle Communication System

SAABIRESH LETCHUMANAN

BACHELOR OF COMPUTER SCIENCE WITH HONOURS

2026

**myMove: IoT Enabled Smart Parking Reservation and
Privacy Vehicle Communication System**

SAABIRESH LETCHUMANAN

**A thesis submitted
in fulfillment of the requirements for the degree of
BACHELOR OF COMPUTER SCIENCE WITH HONOURS**

School Of Computing and Digital Technology

UNIVERSITY MALAYSIA OF COMPUTER SCIENCE AND ENGINEERING

2026

DECLARATION

I declare that this thesis entitled “**myMove: IoT Enabled Smart Parking Reservation and Privacy Vehicle Communication System**“ is the result of my own research, except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Signature: 

Name: SAABIRESH LETCHUMANAN

Date: 11 August 2026

APPROVAL

I hereby declare that I have read this thesis and in my opinion this thesis is sufficient in terms of scope and quality for the award of Degree BACHELOR OF COMPUTER SCIENCE WITH HONOURS

Signature: 

Supervisor: MUHAMMAD SYAHMIE SHABARUDIN

Date:

DEDICATION

To my beloved family, including my wife and parents, I would like to thank them for their continuous love, encouragement, and support throughout my academic journey. Their patience and understanding have been my greatest source of strength during challenging times.

I would also like to dedicate this work to my supervisor and lecturers, whose guidance, advice, and constructive feedback have played an important role in shaping this project. Finally, this thesis is dedicated to all friends on and off campus who offered help, motivation, and support throughout the completion of this study.

ABSTRACT

Parking challenges such as limited space availability, double parking, and inefficient parking management systems are common problems faced by drivers in busy urban areas in Malaysia. Most existing parking applications focus mainly on digital payments and often lack real-time parking information, parking reservation features, and effective communication tools to manage double-parking situations. To address these issues, this project proposes myMove, a smart mobile-based parking platform designed to improve the overall parking experience through a centralized and user-friendly system. The application focuses on features such as viewing parking information, reserving parking spaces, digital payments, booking management, and QR code-based communication between drivers without exposing personal contact details. This study was carried out during the FYP1 phase and involved a literature review and a pilot survey to identify user needs and limitations of existing systems. The findings indicate strong user interest in an integrated parking management solution and provide a solid foundation for the development, implementation, and evaluation of the full system in FYP2.

myMove: Sistem Tempahan Parkir Pintar Berasaskan IoT dan Komunikasi Kenderaan Berprivasi

ABSTRAK

Masalah parkir seperti kekurangan tempat letak kereta, parkir berganda, dan sistem pengurusan parkir yang tidak cekap merupakan isu yang sering dihadapi oleh pemandu di kawasan bandar yang sibuk di Malaysia. Kebanyakan aplikasi parkir sedia ada lebih tertumpu kepada pembayaran digital dan sering kekurangan maklumat parkir masa nyata, ciri tempahan parkir, serta alat komunikasi yang berkesan untuk mengurus situasi parkir berganda. Bagi menangani masalah ini, projek ini mencadangkan myMove, iaitu sebuah platform parkir pintar berasaskan aplikasi mudah alih yang direka untuk meningkatkan pengalaman parkir secara keseluruhan melalui sistem yang berpusat dan mesra pengguna. Aplikasi ini memfokuskan kepada ciri-ciri seperti paparan maklumat parkir, tempahan ruang parkir, pembayaran digital, pengurusan tempahan, serta komunikasi berasaskan kod QR antara pemandu tanpa mendedahkan maklumat peribadi. Kajian ini dijalankan pada fasa FYP1 dan melibatkan kajian literatur serta tinjauan perintis bagi mengenal pasti keperluan pengguna dan kekangan sistem sedia ada. Dapatan kajian menunjukkan minat yang tinggi terhadap penyelesaian pengurusan parkir yang bersepadu dan menyediakan asas yang kukuh untuk pembangunan, pelaksanaan, dan penilaian sistem penuh dalam fasa FYP 2.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor for the continuous guidance, valuable advice, and constructive feedback provided throughout the completion of this Final Year Project. Their support and encouragement were essential in helping me stay focused and motivated during the research and documentation process. I would also like to thank the lecturers and academic staff for sharing their knowledge and providing helpful insights that contributed to the development of this project. Special thanks are extended to all participants who took part in the pilot study survey, as their time, responses, and honest feedback were invaluable in identifying real-world parking challenges and shaping the requirements of the proposed system. Finally, I would like to express my deepest appreciation to my wife, my family, and my friends for their constant support, patience, and encouragement throughout this journey. Their understanding and motivation played a significant role in helping me complete this project successfully.

TABLE OF CONTENTS

DECLARATION.....	
APPROVAL.....	
DEDICATION.....	
ABSTRACT.....	1
ABSTRAK.....	2
ACKNOWLEDGEMENT.....	3
TABLE OF CONTENTS.....	4 - 5
LIST OF TABLES.....	6
LIST OF FIGURES.....	7 - 8
LIST OF ABBREVIATIONS.....	8 - 9
CHAPTER 1	
1.1 Background.....	10
1.2 Problem Statement.....	11
1.3 Research Question.....	12
1.4 Research Objective.....	12 - 13
1.5 Scope of Research.....	13
1.6 Thesis Outline.....	13 - 14
CHAPTER 2	
2.1 Introduction.....	15
2.2 Review of Academic Literature.....	16
2.3 Analysis of Commercial Applications.....	17
2.3.1 Touch ‘n Go eWallet’s parking.....	17 - 18
2.3.2 JomParking.....	18 - 19
2.3.3 Flexi Parking.....	19 - 20
2.4 Suggested Approach.....	20
2.5 Benchmark with Existing Systems.....	21
2.6 Summary.....	22
CHAPTER 3	
3.1 Introduction.....	23
3.2 Proposed System Development Process.....	24 - 26
3.3 Gantt Chart.....	27
3.4 Limitations and Features.....	28
3.4.1 Limitations of Current Systems.....	28
3.4.2 Proposed Features for myMove.....	29
3.5 Data Collection Methods.....	30 - 31
3.6 Develop Module of myMove and Integration.....	32 - 34
3.7 Summary.....	34

CHAPTER 4

4.1 Introduction.....	35
4.2 Functional Requirement.....	36 - 37
4.3 Non-Functional Requirement.....	38 -39
4.4 UI / Design of Application.....	39 - 68
4.5 Backend Implementation.....	69
4.5.1 Backend Architecture.....	69 - 70
4.6 Database Design.....	71
4.6.1 Cloud Firestore Collections Schema.....	72 - 76
4.6.2 Firebase Realtime Database (RTDB) Schema.....	76
4.7 Summary.....	77

CHAPTER 5

5.1 Introduction.....	78
5.2 Project Deliverables.....	78
5.3 Software & Hardware Testing.....	79
5.3.1 Unit Testing.....	79 - 80
5.3.2 Integration Testing.....	80 - 81
5.3.3 System Testing.....	82 - 83
5.3.4 User Acceptance Testing (UAT).....	83 - 85
5.4 Feedback & Discussion.....	85 - 86
5.5 Discussion of Practical Implication.....	86
5.6 System Limitation.....	87
5.7 Summary.....	87

CHAPTER 6

6.1 Introduction.....	88
6.2 Project Summary.....	88 - 89
6.3 Achievement of Project Objectives.....	89 - 91
6.4 Contribution of Systems.....	91 - 92
6.5 System Limitation.....	92 - 93
6.6 Future Enhancement & Recommendation.....	93
6.7 Summary.....	94

REFERENCES.....	95 - 96
------------------------	----------------

LIST OF TABLES

Table	Title	Page
Table 2.1	Comparative Study Of Car Parking System	16
Table 2.2	Comparison Of Existing System	21
Table 4.1	Real-time RTDB Sensor Status Stream Listener	54
Table 4.2	Two-Factor Authentication (2FA) Setup & Secret Generation	55
Table 4.3	Stripe SDK Setup Intent Card Addition Integration	55
Table 4.4	Querying User Booking Transactions from Cloud Firestore	56
Table 4.5	Dynamic System Permissions Handler	57
Table 4.6	Guest Move Request Submission Logic	60
Table 4.7	Real-Time Active Booking Calculations & Listeners	68
Table 4.8	Cloud Firestore users Collection Schema	72
Table 4.9	Cloud Firestore publicVehicles Collection Schema	72
Table 4.10	Cloud Firestore parking_locations Collection Schema	73
Table 4.11	Cloud Firestore parkingSpots Collection Schema	73
Table 4.12	Cloud Firestore bookings Collection Schema	74
Table 4.13	Cloud Firestore chats & messages (Subcollection) Schema	74
Table 4.14	Cloud Firestore chats Collection & messages Subcollection Schema	75
Table 4.15	Cloud Firestore emergencies Collection Schema	75
Table 4.16	Cloud Firestore feedback Collection Schema	76
Table 4.17	Firebase Realtime Database (RTDB) Schema	76
Table 5.1	Unit Testing Summary	79
Table 5.2	Integration Testing Summary	81
Table 5.3	System Testing & Performance Summary	82
Table 5.4	User Acceptance Testing (UAT) Survey Results	84
Table 6.1	Summary of Research Objectives	91

LIST OF FIGURES

Figure	Title	Page
Figure 2.1	Example of Parking System in Touch 'n Go eWallet	17
Figure 2.2	Example in JomParking	18
Figure 2.3	Example of Flexi Parking	19
Figure 3.1	Waterfall Model that will be used for myMove project	24
Figure 3.2	Gantt Chart of Project Timeline	27
Figure 4.1	Use Case Diagram	37
Figure 4.2	myMove App Initialization & Splash Screen	39
Figure 4.3	User Authentication (Login and Registration Screens)	40
Figure 4.4	Main Dashboard & Parking Locations	41
Figure 4.5	Parking Location Details, Pricing Structure and Select Date & Time	42
Figure 4.6	Real-time Parking Grid with Indicators	43
Figure 4.7	Booking Reservation Summary and Checkout Interface	44
Figure 4.8	Manage Bookings and More Info Page	45
Figure 4.9	Extend Parking and Successful Page	46
Figure 4.10	SOS / Emergency Panic Alert Trigger Overlay	47
Figure 4.11	In-App Real-time Chat	48
Figure 4.12	QR Code Scanner and Display for Vehicle Identification / Chat	49
Figure 4.13	Main Settings & Edit Profile / Vehicle Page	50
Figure 4.14	Account Settings and Two-Factor Authentication (2FA)	51
Figure 4.15	Payment Settings / Stripe Payment Method and Payment History	52
Figure 4.16	Push Notification Settings and Permissions Access Settings	53
Figure 4.17	Webpage for Blocked Vehicle QR Scan and Photo Proof Verification	58
Figure 4.18	Successful Request Notification and Web-Based Chat	59
Figure 4.19	Admin Panel Login	61
Figure 4.20	Admin Dashboard with Analytics and Weekly Revenue	62
Figure 4.21	Parking Location Management Panel	62
Figure 4.22	Manage Parking Spot	63
Figure 4.23	Booking Management Monitor	64
Figure 4.24	Broadcast Notification to All Users	65

Figure 4.25	User Acceptance Testing Feedback Page	66
Figure 4.26	SOS / Emergency Panic Trigger Alarm	67
Figure 4.27	myMove BaaS Architecture Diagram	69
Figure 4.28	ER Diagram of Cloud Firestore Database Schema	71
Figure 5.1	Unit Testing Execution Summary by Category	80
Figure 5.2	Integration Testing Execution Summary by Path	81
Figure 5.3	Response Time Graph in Seconds	83
Figure 5.4	Overall User Acceptance Testing (UAT) Results from Q1 - Q8	85

LIST OF ABBREVIATIONS

Abbreviation	Full Term
2FA	Two-Factor Authentication
ANPR	Automatic Number Plate Recognition
API	Application Programming Interface
APK	Android Application Package
BaaS	Backend as a Service
BLE	Bluetooth Low Energy
CRUD	Create, Read, Update, Delete
DBKL	Dewan Bandaraya Kuala Lumpur
ERD	Entity Relationship Diagram
ESP32	Espressif Systems 32-bit Microcontroller
FCM	Firebase Cloud Messaging
GPS	Global Positioning System
HC-SR04	Ultrasonic Distance Sensor Module
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
IoT	Internet of Things
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
LED	Light Emitting Diode

LPR	License Plate Recognition
MYR	Malaysian Ringgit
NoSQL	Non-Relational SQL / Not Only SQL Database
OTP	One-Time Password
P2P	Peer-to-Peer
QR	Quick Response Code
REST	Representational State Transfer
RFID	Radio Frequency Identification
RTDB	Firebase Realtime Database
SDK	Software Development Kit
SDLC	Software Development Life Cycle
SOS	Emergency Panic Distress Signal
SSL	Secure Sockets Layer
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
UAT	User Acceptance Testing
UI	User Interface
UID	Unique Identifier
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1

INTRODUCTION

1.1 Background

In recent years, searching for parking has become increasingly stressful because people who own vehicles are rising day by day (Chen et al., 2025). High density areas such as condominiums, shopping malls and office or university campuses face these issues endlessly. Facilities like visitor parking, resident parking and loading bays must operate efficiently to ensure smooth traffic flow and reduce congestion (Alshareef., 2024). However, issues such as double parking, difficulty finding parking spots, and long waiting times often cause frustration for drivers. Looking at today's technologies, many individuals must have their own mobile phone that allows them to access the internet or download numerous applications (Khalid et al., 2024).

Despite improvements in today's tech, many parking areas still depend on manual checking, simple ticketing systems, and using physical cards that do not provide real-time information (Puspitasari et al., 2021). These systems often lack important functions like digital reservations, live availability updates, and cashless payments. As a result, people spend unnecessary time searching for parking in the same area, leading to waste of time and stress. To address these challenges, the most important aspect of this project is to create a user-friendly mobile application named myMove that allows users to check availability parking, reserve parking, make digital payment, and manage double-parking situation through QR Code and in-app communication that will satisfy customer's needs.

1.2 Problem Statement

The approach to urban parking management these days encounters several significant difficulties in today's rapidly evolving cities. Drivers struggle to locate available parking spaces due to outdated systems that do not provide real-time availability or clear guidance on where to park (Kalašová et al., 2021). For example, a typical parking area in Malaysia which forces drivers to continuously circle the premises in search of an unoccupied parking spot, leading to inefficiency and congestion. When drivers keep searching for a parking spot even though none are available, it causes stress and wastes their time, which can reduce their overall productivity. Besides, many parking systems in Malaysia are outdated and cannot keep up with the growing number of vehicles. Because they do not provide real-time information, drivers spend more time searching for parking, which increases traffic congestion and reduces the overall quality of urban mobility.

Additionally, double-parking remains a persistent issue as well in Malaysia, especially in crowded areas, where blocked vehicle owners have no fast or secure way to contact the driver who parked behind them (Haslinda et al., 2016). Despite DBKL's implementation to clamp cars that park in illegal parking areas in 2016, recent studies show that the number of compounds issued has increased dramatically between January to May 2025, which DBKL also received 1,009 public complaints related to illegal parking (Anis, 2025). This shows that current enforcement methods alone are not enough to solve the ongoing illegal and double-parking issues.

1.3 Research Question

This study aims to answer the following research questions:

- i. How can a mobile application improve the way drivers find and reserve parking spaces without relying on physical hardware?
- ii. How does QR-based communication help drivers handle double-parking situations?
- iii. Can real-time applications reduce search time for drivers in busy urban areas?
- iv. Can a dedicated application like myMove effectively solve the problems in real world situations and existing platforms?

1.4 Research Objective

The aim of this project is to develop a parking system that provides real-time information, finds parking spaces easily, reserves a spot and communicates during double-parking situations to reduce stress, search time and congestion.

The objectives of the project are as follows:

- i. To study existing parking systems, identify current issues faced by Malaysian drivers, analyze the limitations of current car parking application, define user requirement specification and features through literature review and survey
- ii. To develop the myMove application (mobile and web admin) by implementing its core features, including real-time parking search, hardware-integrated spot monitoring, and booking.

- iii. To evaluate the application usability and user satisfaction by conducting User Acceptance Testing (UAT) with real users and collecting feedback for further improvement.

1.5 Scope of Research

The scope of this research is as follows:

- i. Development of a mobile application (Android) using Flutter and backend with Firebase for authentication.
- ii. This application targets Malaysian drivers who had overwhelming and frustrating experiences during their daily search for parking or dealing with double-park situations.
- iii. Users will be able to create an account, which prompts them to enter details such as name, age, email, password, car number plate

1.6 Thesis Outline

Based on the objectives previously presented and, on the approach proposed before, this thesis is made up of five (5) chapters, which contents are summarized as follows:

- Chapter 1 (Introduction)
 - This chapter presents an overview of the project, including the background of the study, problem statement, objectives, scopes, contributions and significance of the research.

- Chapter 2 (Literature Review)
 - This chapter provides a review of existing literature related to the project and mobile parking applications. This chapter examines the current applications such as Touch ‘n Go eWallet, JomParking and Flexi Parking.
- Chapter 3 (Methodology)
 - This chapter explains the research methodology applied in this study. It outlines the research approach, data collection methods and the software development process that will be applied for this project.
- Chapter 4 (Result and Discussion)
 - This chapter presents the results and findings obtained from the pilot study and prototype development of this project. It includes survey results, user feedback and analysis related to current parking issues and user expectations.
- Chapter 5 (Conclusion and Recommendation for Future Search)
 - This chapter concludes the study by summarizing the main findings and contributions of the research. It also discusses the limitations of the current study and provides recommendations for future work, particularly for the full system development and evaluation in FYP2.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

In recent years, technological advancements have significantly transformed various aspects of daily life (Aditya Kumar 2022). In this chapter it reviews existing research, studies and technologies related to mobile applications used for parking management, real-time parking detection and driver communication. The review identifies the current challenges faced by Malaysian drivers, including double parking, outdated parking system / application and lack of real-time parking availability. It examines current solutions like Touch 'n Go eWallet's in-app parking, JomParking and Flexi Parking to identify their strength. This review establishes the need for a more modern and integrated parking management system that offers features such as real-time parking availability, parking reservation, digital payments, automatic number plate detection, in-app communication between drivers, push notifications for car-blocking alerts, navigation to available parking lots, and a QR-based contact system for emergencies. Together, these features aim to improve driver convenience, reduce parking-related conflicts and modernize the overall parking experience in Malaysia.

2.2 Review of Academic Literature

To better understand the current landscape of mobile based parking management and communication systems, three relevant studies and existing solutions were reviewed. These studies helped identify key design features and gaps that inform the development of myMove.

Table 2.1 Comparative Study Of Car Parking System

Source	Main Findings	Methodology	Strengths	Weaknesses	Relevance to Project
P. Sampath, et al. IoT- based Smart Parking System for Industry 4.0 with QR code access and real- time navigation (2025)	The study shows that using IoT sensors, QR codes, and a mobile app can reduce parking search time, congestion, and user frustration, while improving parking efficiency and security.	IoT sensors, QR codes, and a mobile app can make parking faster, safer, and less stressful.	Offers real-time parking info, QR-based access, reservations, navigation, and a working hardware-software prototype.	The system is more suitable for controlled environments like malls or campuses but difficult to scale without infrastructure support.	This study highlights myMove's key features: real-time parking info, reservations, QR-based interaction, and mobile app use.
Mahat, S. N. & Nadzir, N. M. "Assessing User Satisfaction of Smart Parking Payment System in Selangor" (2025)	User satisfaction in parking apps mainly depends on how easy they are to learn and how clearly information is shown, making usability more important than having many complex features.	A survey conducted among local users with statistical analysis to identify the factors that influence their satisfaction.	Uses data from Malaysian users to provide clear insights and recommendations on user experience.	Focuses on payment features rather than driver-to-driver communication or QR code sticker interactions.	Supports the importance of having a simple user experience, clear information, and features adapted in MyMove.
Balqis Lim "Uni students develop smart system to curb double parking problems (ParkKing)" (2017)	A smart outdoor parking system aimed at reducing double-parking by detecting illegally parked vehicles and enabling more efficient parking supervision	The project developed a ParkKing prototype using sensors to detect and manage double-parking in outdoor lots.	The system helps reduce double-parking by identifying problem parking and using technology to support enforcement and reduce traffic issues.	It lacks technical details such as system accuracy, implementation scope, real-world testing, and driver communication, focusing more on the announcement than evaluation.	This study shows a real-world effort to address double parking and highlights the need for better parking systems. It supports myMove's use of technology to manage illegal parking and shows the potential of smart detection systems in parking solutions.

2.3 Analysis of Commercial Applications

This section explains about the functionality and limitations of existing platforms used by Malaysian drivers. Below are three applications, Touch 'n Go eWallet's parking, JomParking and Flexi Parking were chosen because of their popularity.

2.3.1 Touch 'n Go eWallet's parking

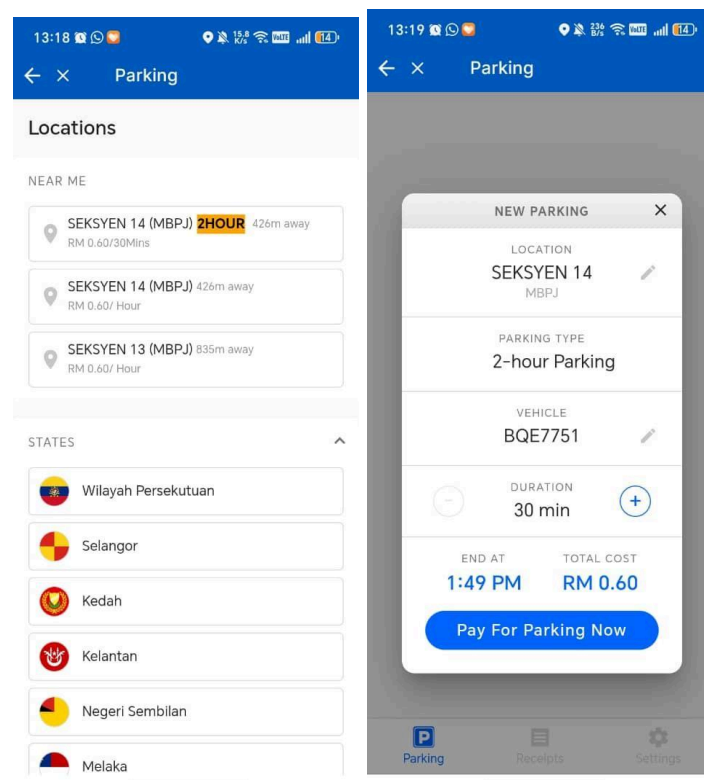


Figure 2.1: Example of Parking System in Touch 'n Go eWallet.

Touch 'n Go eWallet parking is a feature inside the Touch 'n Go eWallet app that allows users to pay for parking digitally only at selected locations / street parking. The app is convenient because there is no need to use physical tokens, it automatically deducts parking fees from their eWallet balance and has LPR Parking feature. However, there are certain features that have limitations and lack features. It does not show real-time parking availability, so drivers still need to drive around to find an empty spot.

2.3.2 JomParking

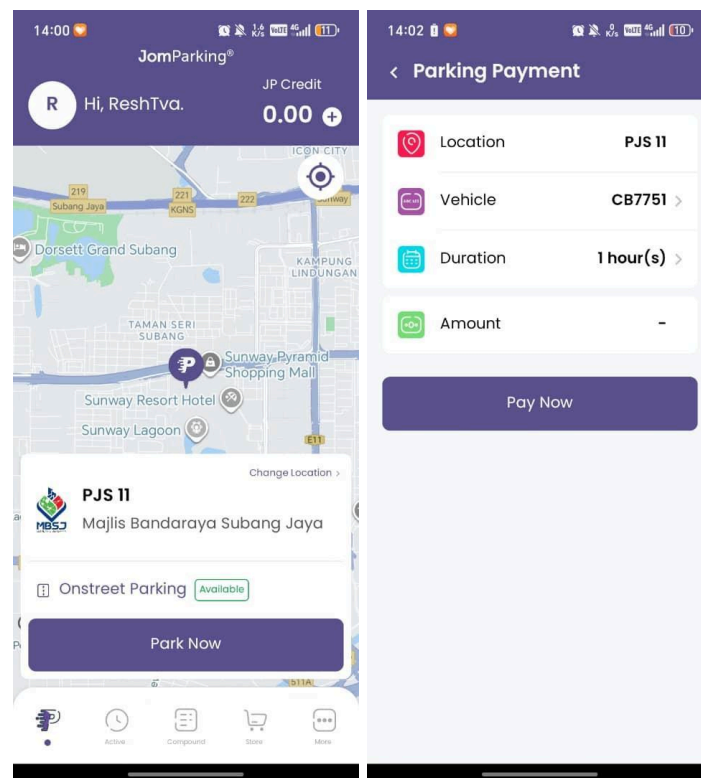


Figure 2.2: Example in JomParking

JomParking is a mobile parking application widely used in Malaysia as Touch 'n Go eWallet to pay for street and city council parking. The app allows users to select their location by dragging the map, entering their car number plate and selecting the duration of the parking time. It also provides a city council compound, where users can pay for outstanding payments and includes parking passes, reload credit to the app and services like car battery delivery, booking a trip and car and motorbike insurance. However, their platform is basic and offers limited interaction. It doesn't support every city council parking around Malaysia and doesn't include every off-street parking like malls or buildings.

2.3.3 Flexi Parking

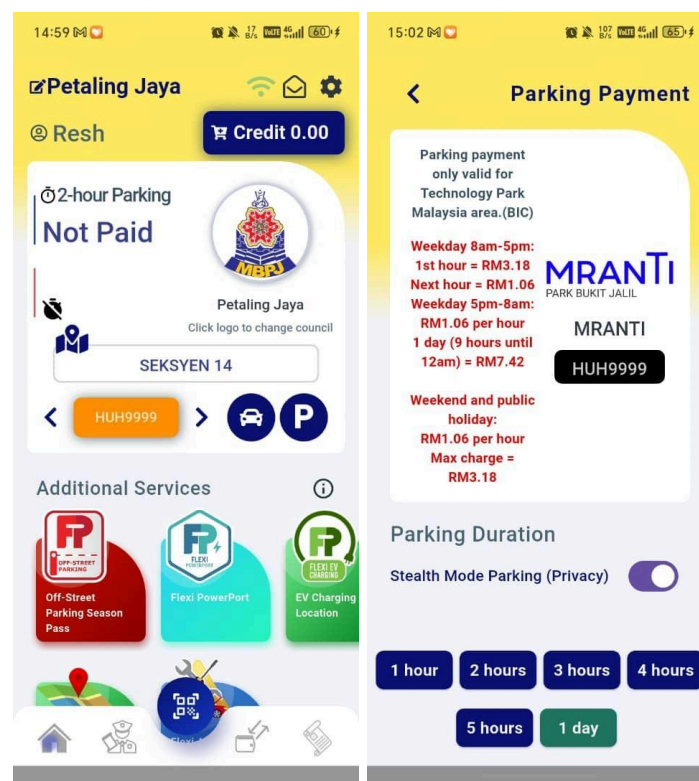


Figure 2.3: Example of Flexi Parking

Flexi Parking is a local platform in Malaysia that offers to pay for street parking under supported city councils. Users can select their parking zone, add durations, their car number plate and pay using in-app credit. However, it's not fully operational and optimized in terms of parking experience. The platform sometimes runs into an error, like connection issues, payment and top out problems, and receiving suspicious messages about traffic fines.

2.4 Suggested Approach

The proposed system is a mobile application called myMove, designed to improve the parking experience for Malaysian drivers by providing clearer information, easier communication and a more organized parking process. myMove focuses on giving drivers real-time parking availability with information, a reservation system, simple digital payment and a faster way to deal with double-parking.

The system allows users to view available parking spots in a building, select a space with details such as floor and parking number, and reserve it before entering. Once a spot is chosen, it becomes unavailable to others, reducing confusion and unnecessary searching around the same place. The app also sends reminders when the user's parking time is ending to help avoid extra charges. Additionally, a QR-based double-parking communication feature let blocked drivers scan a displayed QR Code to contact the car owner through in-app messaging or calls without revealing personal phone numbers.

2.5 Benchmark with Existing Systems

The comparative analysis of existing systems and the proposed myMove application is detailed in the table below:

Table 2.2: Comparison Of Existing System

App Name	Functionality	Strengths	Weaknesses	Proposed Improvements
Touch 'n Go eWallet	It has parking features, such as LPR parking, street parking and provides travel passes, named My50 Pass.	Convenient and fast payment method while using LPR parking and reduces physical cash or card.	The balance in the physical card and eWallet balance lacks integration and support limited location only.	Aligns with myMove goal and expands to more locations.
JomParking	Payment for street parking by providing city council's, allow by user by dragging the map and offer compound payment.	Convenient and time-saving features like paying for parking remotely and extending their time.	Supports only limited locations, provides only certain compound payment and only has online banking.	Aligns with myMove goal to show parking details and parking space selection
Flexi Parking	Users can pay for on-street parking via GPS, receive expiry reminders, extend parking remotely, and manage up to 10 vehicles in one account.	Supports over 46 municipal councils, monthly parking passes and prevents users from paying during non-chargeable hours.	App hard to navigate, buggy, errors when registration, provides certain locations and inaccuracy GPS	Aligns with myMove goal to show parking details and parking space selection

2.6 Summary

This chapter examined existing parking applications used in Malaysia to identify their features, strengths and weaknesses. Applications such as Touch ‘n Go eWallet Parking, JomParking and Flexi Parking were reviewed and found to mainly focus on digital payment while it lacks comprehensive parking information, provides limited coverage, minimal user interaction and lack of support for double parking situations. These findings justify the need for the proposed myMove application as more integrated and user-friendly parking solutions.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter outlines the research methodology that addresses the Objective 1 of myMove project, which focuses on examining the limitations of current car parking applications, studying existing parking solutions and identifying user needs before developing the proposed system. This structured research approach is adopted to ensure that the problems are clearly understood, and system requirements are correctly defined. For this project, the Waterfall Model is selected as the software development methodology because of its suitability for systems with well-defined requirements and a predictable development path. This model follows a linear structure in which each phase is completed thoroughly before moving on to the next part. The research design for Objective 1 includes a detailed literature review and a simple user survey to study existing parking applications, identify frustrations and analyze gaps in the current system. Later phases including system design, development, usability testing, deployment and maintenance will also be planned, followed by the Waterfall methodology.

3.2 Proposed System Development Process

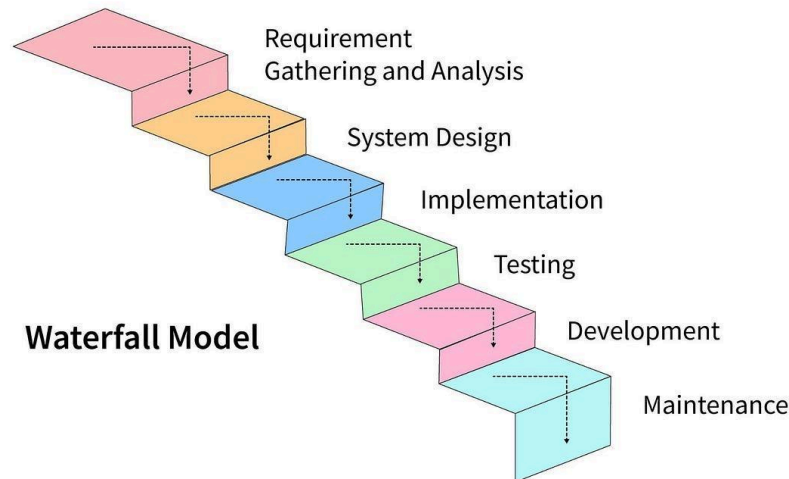


Figure 3.1 Waterfall Model that will be used for myMove project

This project will be developed using the Waterfall Model, which follows step-by-step sequence from the initial study of requirements to the final maintenance of the system. The model is chosen for myMove because the requirements are clearly defined, stable, and suitable for a step-by-step development approach, allowing each phase to be carried out with clarity and minimal doubt. By completing one phase fully before moving to the next, the development process becomes more manageable, organized and easier to work with. Below shows how each stage of the Waterfall Model will be applied throughout the system development:

- Requirements Phase:
 - This phase is completed in FYP1, all information related to the parking problems, user needs, and system expectations is gathered and analyzed. This includes studying existing solutions, identifying limitations such as absence of real-time parking availability, lack of information and limited support for handling double parking situations. Feedback gathered through pilot study is used to define the functional and non-functional requirements that will guide system design and development in the next phase.

- Design Phase:
 - The design phase focuses on turning the identified requirements into a clear system plan. This phase includes the creation of use-case diagrams and UI prototypes to show how users will interact with the application. The UI designs developed in FYP1 prioritize simplicity and ease of use to demonstrate the main workflow, while more detailed and complete interface designs will be developed in FYP2.

- Development Phase:
 - The development phase will take place in FYP2, where the myMove application will be built based on the approved system design. Core features will be developed step by step, focusing on creating modular and maintainable code. This phase will also involve integrating the frontend and backend components to ensure smooth data flow and proper system functionality.

- Testing Phase:
 - Testing will be carried out throughout FYP2 to ensure the system works as expected and meets user requirements. Functional testing will be conducted to check that each feature operates correctly, while basic usability testing will be used to assess user interaction and interface clarity. Any issues or errors identified will be resolved before the system is deployed.

- Deployment Phase:
 - After development and testing are completed, the application will be deployed in a controlled environment for demonstration and evaluation. This phase allows the system to be reviewed as a fully working prototype and prepares for further improvement through user experience.

- Maintenance Phase:
 - The maintenance phase focuses on making improvements and refinements after the system is deployed. Feedback gathered during testing and evaluation will be used to fix issues, improve performance and enhance system features. This phase also considers system scalability and the addition of new functionality in future versions of the application.

3.3 Gantt Chart

Activity	September	October	November	December
Problem Statement & Topic Approval	✓			
Literature Review	✓	✓		
Pilot Study & Survey Data Collection		✓	✓	
Data Analysis & Findings			✓	
System Requirements			✓	✓
Use Case Diagram			✓	✓
UI Prototype Design			✓	✓
FYP1 Thesis Submission				✓

Figure 3.2 Gantt Chart of Project Timeline

3.4 Limitations and Features

To establish a foundation for myMove's development, the limitations of the current car parking system and the desired features for the mobile application were identified through a literature review and pilot study findings.

3.4.1 Limitations of Current Systems

- **Lack of Mobile Accessibility:** Many malls and buildings rely on outdated systems that are not integrated with mobile applications, making it inconvenient for people to search for their parking or view how much time they spend.
- **Lack of Hardware Integration:** With parking availability relying on manual updates or no hardware system in place.
- **Missing Parking Info:** Current application doesn't show building name, floor and parking space number
- **Absence of Double-Parking Solutions:** Current systems do not provide any built-in mechanism to handle double-parking situations, leaving drivers without a secure and efficient way to contact other vehicle owners.
- **Lack of Integrated Communication Features:** Existing systems do not support in-app messaging and calling features for double-parking situations, forcing users to rely on external methods that may compromise privacy.

3.4.2 Proposed Features for myMove

Based on the pilot study with 44 people, that include university students and other individuals, the following features were identified as crucial for myMove to address these limitations:

- **Real-Time Parking Availability:** Allowing users to view available and occupied parking spaces in buildings and help drivers make faster parking decisions.
- **Parking Reservation:** Enabling users to reserve specific parking spaces in advance to reduce unnecessary searching.
- **Navigation Assistance:** Providing guided directions to the selected parking area to improve user and reduce travel time .
- **Parking Details:** Displays information such as parking fee rate, building name, floor level, parking space number and parking duration once the car is parked.
- **Emergency Mode:** Providing quick-access features that allow users to send urgent alerts in emergency situations.
- **Space for User Feedback:** Receive feedback from users / drivers about their experience in the specific place where they park so that we can improve.
- **QR Code-Based Solution for Double Parking:** Allow blocked drivers to scan a QR Code to contact car owners quickly without exposing confidential details.
- **In-App Messaging and VoIP Calling:** Enable secure communication between drivers through chat, and voice call within the application.

3.5 Data Collection Methods

To investigate the challenges faced by drivers and to gather user requirements for the parking system, the following methods were used:

Literature Review:

- **Source:** Existing mobile parking applications and publicly available documentation, including Touch 'n Go Parking, Flexi Parking, and JomParking, were reviewed to understand current practices and system capabilities.
- **Focus:** The review focused on identifying common limitations in existing systems, such as lack of real-time parking availability, poor user communication features, and limited integration between parking information and user interaction tools.
- **Process:** Information was gathered through feature analysis of existing applications, official app descriptions, user reviews, and observational evaluation conducted between 2020 and 2025. A comparative analysis was then performed to identify system gaps and areas for improvement.

Pilot Study:

- **Participants:** A total of 37 participants including university students, and working adults aged between 18 to 45 above have taken part in the online survey to share their experiences and consent about car parking systems
- **Method:** Google Form was used to collect responses on difficulties finding parking spaces, delays caused by blocked vehicles and user expectations for a smart parking application.
- **Data Collection:** The results indicate that most respondents had trouble in finding available parking spaces (56.8%) and encountered double parking situations (89.2%). Most participants expressed strong interest in features such as showing vehicle parking details (73%), digital payment integration (62.2%) and QR code that helps blocked drivers (51.4%) as a main feature.

3.6 Develop Module of myMove and Integration

While the development phase will take place in FYP2, this section outlines the approach for bringing together myMove core components based on the requirements gathered in Objective 1. myMove will be developed as a cross-platform mobile application using Flutter with Dart for the frontend. Although Flutter supports both Androids and iOS, the first release of myMove will be limited to Android devices only due to current hardware and incompatibility with Apple devices. The interface will be designed using Figma, focusing on clean and minimal layout and user-friendly experience.

The integration plan covers:

- **Frontend Development:**

Flutter will be used to develop an intuitive and responsive user interface compatible with Android devices. The frontend will include important key features such as:

- User login / registration (with secure authentication)
- Displays an interactive map for exploring nearby parking locations.
- A search bar for destinations and a bottom navigation to profile, QR scanner for double parking situation and history
- Shows a building-level parking layout with color-coded indicators for available, occupied, and reserved spaces.
- Users can select a parking spot, and the system updates its status when they arrive and extend parking duration
- Shows parking details (floor, section, lot, duration), lets users upload a photo, and provides fee info with instant payment.

- **Backend Development:**

The backend will be powered by Firebase that includes various product categories like authentications and more. The core backend responsibilities include:

- Managing authentication (email, password, or Google sign-in) using Firebase Authentication
- Deliver instant push notification using Firebase Cloud Messaging (FCM)
- Manage real-time parking status updates using Firebase Realtime Database
- Support media uploads such as parking spot photos using Firebase Storage
- Location based parking search and navigation guide using Google Maps API
- Server-side processes such as payments and reservations using Cloud Functions.
- Generate and scan QR codes using Flutter QR libraries (qr_flutter)
- Communication and calls using Firebase Realtime DB and third-party API's
- Payments will be handled using Stripe, Touch 'n Go eWallet, or any supported Malaysian gateway integrated through Firebase Functions.

- **Database Structure:**

- Storing user details, building, floor and parking lot data for map and location retrieval using Firebase Firestore
- Manage communication records (chat messaging and call) using Agora

- **Hardware Integration:**

- Parking sensors will be installed in individual parking spaces to detect whether a spot is occupied or vacant using ESP32 microcontrollers.
- Each ESP32 device will transmit parking status data wirelessly to the backend system through Wi-Fi and sync with Firebase Firestore.

3.7 Summary

This chapter describes the methodology used for the myMove project, where the Waterfall Model was applied to guide system development in a clear and structured way. It describes how user needs and system limitations were identified through a literature review and pilot study carried out during FYP1. The chapter also presents the proposed features and overall system, including frontend, backend, database and hardware. In addition, a Gantt chart is included to show the planned project timeline and development schedule.

CHAPTER 4

DEVELOPMENT & IMPLEMENTATION

4.1 Introduction

This chapter presents the technical design, system architecture, hardware integration, database design, and software implementation of myMove: IoT Enabled Smart Parking Reservation and Privacy Vehicle Communication System.. Following the Waterfall method approach introduced in Chapter 3, this phase transforms the system requirements into a functional smart parking solution. The objective of this phase is to integrate IoT hardware, cloud services, and mobile applications into a unified platform that enables real-time parking occupancy monitoring, parking reservations, secure digital payments, and efficient parking management. The system consists of three main layers: the IoT Hardware Layer, the Cloud Backend Layer, and the Application Layer. The hardware layer uses an ESP32 microcontroller with HC-SR04 ultrasonic sensors to detect parking slot occupancy. The cloud backend is powered by Firebase, utilizing Firebase Realtime Database for real-time sensor updates, Cloud Firestore for structured data storage, and Firebase Authentication for secure user management. At the application layer, myMove provides a Flutter-based mobile application that allows users to view parking availability, reserve parking spaces, make digital payments, communicate with vehicle owners through QR codes, and access emergency SOS features. A responsive web dashboard enables administrators to monitor parking occupancy, manage parking facilities, and track emergency alerts. This chapter also presents the system's functional and non-functional requirements, system architecture, hardware implementation, database design, and key software

components developed during the implementation phase. Finally, it summarizes the development milestones achieved.

4.2 Functional Requirement

This section outlines the functional requirements of the myMove, derived from user needs gathered during the requirement analysis phase.

- **User Authentication and Session Management:** Secure user registration, login, profile management, and persistent user sessions in the Flutter mobile application using Firebase Authentication.
- **Vehicle & QR Code Management:** CRUD management of registered vehicles (license plate, model, color) and automatic generation of unique, high-resolution contact QR codes stored and retrieved via Firebase Storage.
- **Real-time Parking Spot Grid:** Dynamic, visual grid layout representing available and occupied spots. The grid updates immediately in response to live sensor states utilizing stream listeners connected to the Firebase Realtime Database (RTDB).
- **Parking Spot Reservation & Booking:** Spot selection, time duration setup, price breakdown calculation, and confirmation of booking status persisted in Cloud Firestore.
- **Payment Gateway Integration:** Secure processing of credit card transactions for reservations using Stripe API and SDK integrations.
- **Ultrasonic Occupancy Detection:** Proximity monitoring of physical parking spots using HC-SR04 ultrasonic sensors controlled by an ESP32 microcontroller to identify vehicle presence.
- **Local Status Feedback:** Hardware status indicators using dual LEDs connected to the ESP32 (Red for occupied slots, Green for vacant slots) to provide immediate feedback to drivers in the physical parking lot.
- **Live Database Synchronization:** Automatic transmission of physical slot changes to the cloud from the ESP32 utilizing HTTP requests/REST APIs connected to the Firebase RTDB.
- **SOS & Emergency Broadcast:** One-tap emergency/panic trigger in the mobile app, instantly transmitting user profile details and current GPS coordinates to the Admin Dashboard for operator action.
- **Web Guest Contact Flow:** Web-based portal (scanner-web) allowing non-app users (guests) to scan a vehicle's QR code, verify the vehicle's public details, and request a vehicle move.
- **Blocked-Vehicle Alerts & Move Requests:** Ability to upload proof photos and share GPS coordinates via the web guest browser to report blocking vehicles.

- **Real-Time Anonymous Chat:** Seamless, real-time messaging between a blocked driver (using the web guest portal) and the blocking car owner (using the Flutter in-app chat) to coordinate relocation without exposing personal contact details.
- **Web Admin CRUD Portal:** Administrative management allowing operators to create, read, update, and delete parking locations, zones, and individual spot-to-sensor mappings via the Web Admin Dashboard (admin-web).
- **Emergency Alert & Live Operations Hub:** Real-time console for admins to monitor triggered SOS emergency alerts, review active bookings, and broadcast system-wide announcements to all mobile app users.
- **User Feedback and Bug Logging:** Submission and management of user experiences, rating logs, and issue reports within the system database.

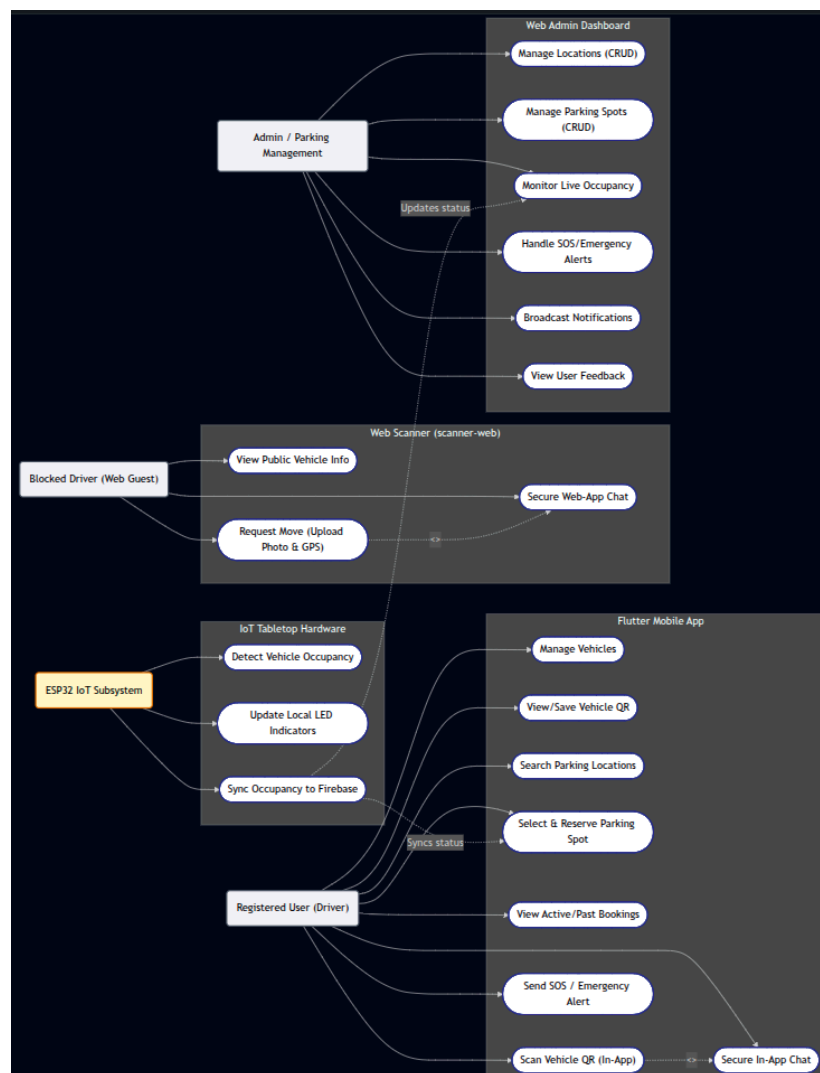


Figure 4.1: Use Case Diagram

4.3 Non-Functional Requirement

- **Security & Data Privacy:**

- Secure password hashing and token-based session management using Firebase Authentication. Access controls are enforced via custom Cloud Firestore Security Rules to verify that users can only read/write their own profiles, vehicles, and bookings. Contact between users is anonymous; phone numbers and emails are never exposed, and communication is routed securely via a vehicle's public QR identifier.

- **Performance & Real-Time Latency:**

- Real-time state synchronization of parking spot occupancy from the physical ESP32 sensor to the cloud database (Firestore RTDB) and onto the driver's Flutter UI must reflect in under 1.5 seconds. End-to-end chat message delivery between the web guest scanner and the mobile application must execute in less than 1.0 second under normal network conditions.

- **Web Guest Optimization:**

- The Web Guest scanner portal (scanner-web) must load and become interactive on mobile browsers in under 2.0 seconds without requiring heavy assets, registration, or native app installation.

- **Reliability & Hardware Fault Tolerance:**

- The ESP32 microcontroller firmware implements a self-healing Wi-Fi and Firestore reconnection loop to automatically recover from wireless dropouts without manual hardware resets. Sensor data is stabilized using a multi-sample averaging/debounce algorithm to prevent false occupancy triggers from transient obstructions (e.g., passing pedestrians).

- **Transactional Integrity (Concurrency):**

- Parking reservations must be processed using Firestore Transactions to guarantee atomic operations and prevent double-booking of a single slot by simultaneous users.

- **Scalability & Database Efficiency:**

- The database schema partitions parking spots hierarchically by location_id and zone_id in Cloud Firestore and Firestore RTDB, ensuring horizontal scalability to support thousands of spots without query latency degradation. Query performance is maintained through optimized database indexing.

- **Usability & Layout Responsiveness:**

- All web frontends (scanner-web and admin-web) are built with responsive layouts adapting dynamically to mobile, tablet, and desktop viewports. The blocked driver reporting workflow is designed as a quick three-step process (Scan QR -> Upload Photo/GPS -> Start Anonymous Chat) to minimize user friction.

- **Compatibility & Platform Support:**

- The Flutter mobile application is compiled to support Android (API Level 21 / Android 5.0 and above) and iOS (iOS 12.0 and above), while the web interfaces are fully compatible with Google Chrome, Apple Safari, Mozilla Firefox, and Microsoft Edge.

4.4 UI / Design of Application

- **Part 1: Mobile Frontend (Flutter App)**



Figure 4.2: myMove App Initialization & Splash Screen

Figure 4.2 shows the initial application loading sequence and splash screen displayed upon opening myMove. The splash screen initializes core application dependencies, verifies local session tokens, and smoothly transitions the driver to either the login page or the main dashboard based on authentication state.

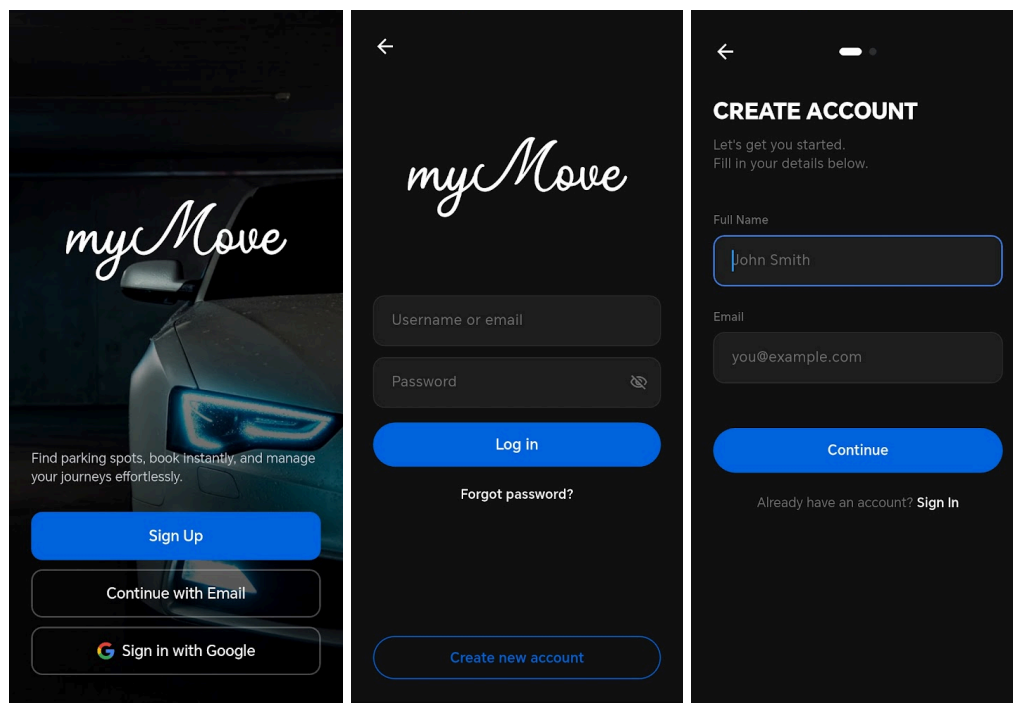


Figure 4.3: User Authentication (Login and Registration Screens)

Figure 4.3 presents the user authentication interface, featuring credential-based login and new user registration flows. Users can log in using their registered email and password, or perform instant single sign-on via Google OAuth. The registration form includes real-time input validation for password strength and user profile creation.

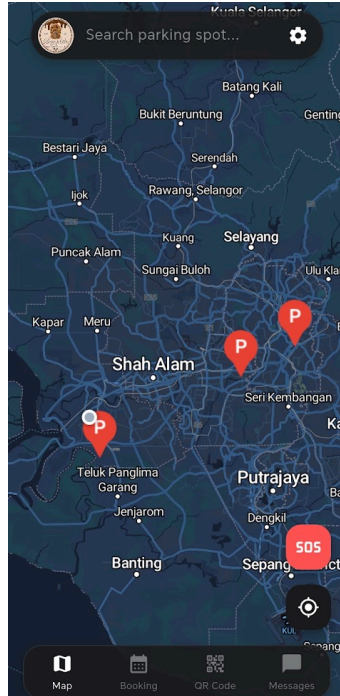


Figure 4.4: Main Dashboard & Parking Locations

Figure 4.4 displays the main user dashboard, serving as the central hub for discovering nearby parking facilities. It provides a real-time list of available parking locations, search filtering by name or proximity, quick navigation shortcuts, and active booking summary cards.

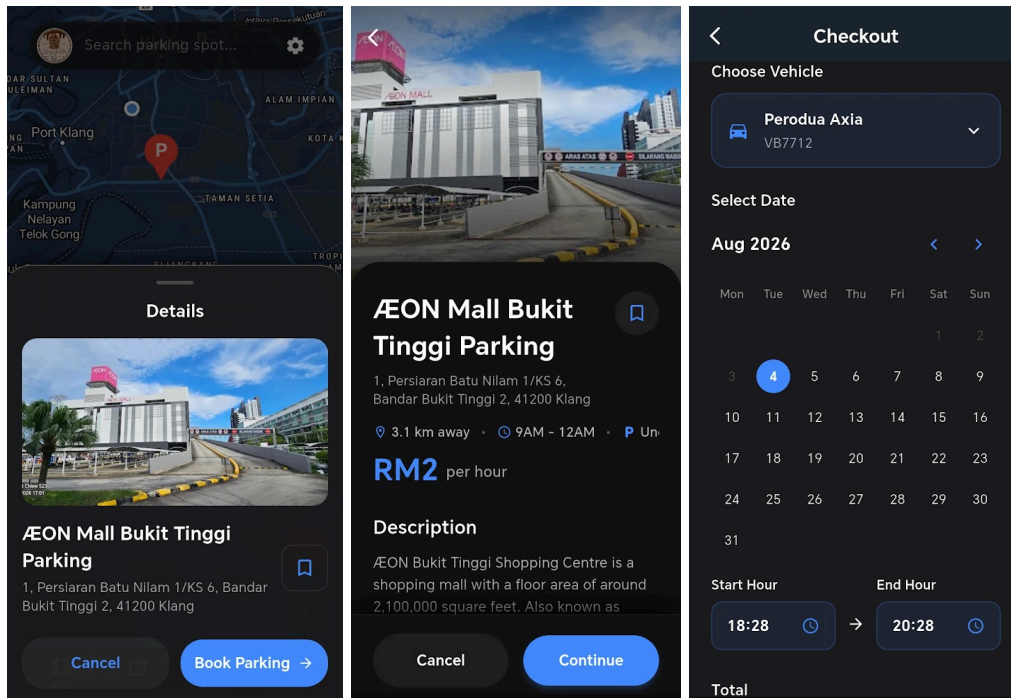


Figure 4.5: Parking Location Details, Pricing Structure and Select Date & Time

Figure 4.5 shows the facility details page, displaying operating hours, hourly rates, amenities, and slot availability. Users can select their desired parking entry date, start time, and reservation duration through an interactive date-time picker prior to spot selection.

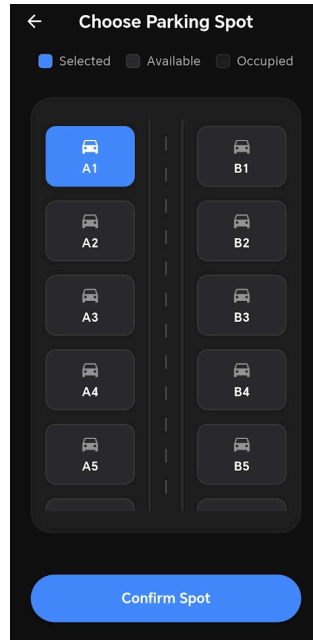


Figure 4.6: Real-time Parking Grid with Indicators

Figure 4.6 illustrates the real-time interactive parking grid layout. The interface displays parking spots color-coded by state, dark grey for available spots, light grey for occupied spots synced live from the ESP32 sensors via Firebase RTDB, and blue for the currently selected spot.

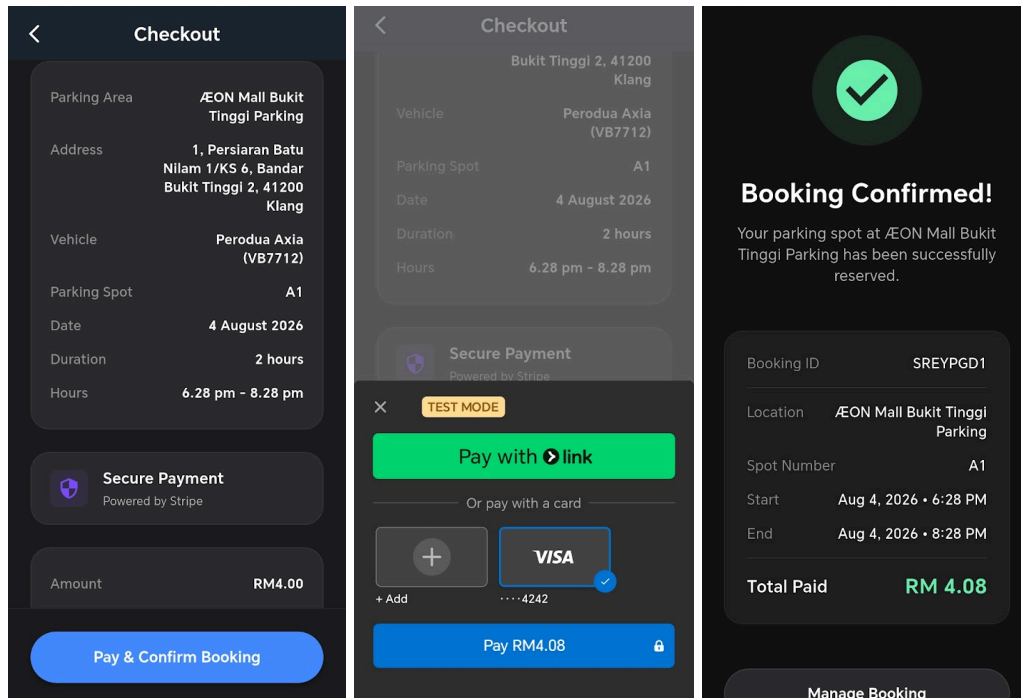


Figure 4.7: Booking Reservation Summary and Checkout Interface

Figure 4.7 shows the booking summary and checkout confirmation screen. It presents a complete financial breakdown including selected spot ID, duration, rate, taxes, and vehicle selection, integrated directly with the Stripe SDK Payment Sheet modal for secure card authorization.

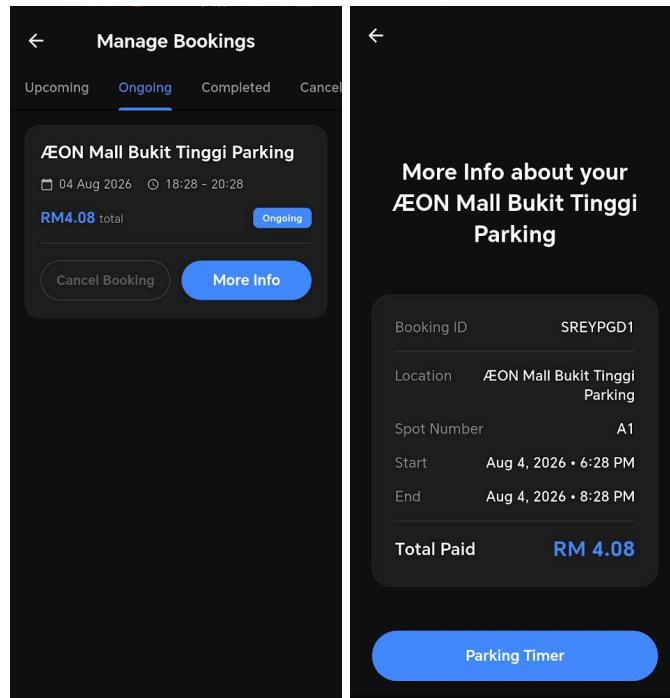


Figure 4.8: Manage Bookings and More Info Page

Figure 4.8 shows the active bookings management screen. Drivers can view their active, upcoming, and completed reservation history, access digital entry pass details, and inspect parking location directions.

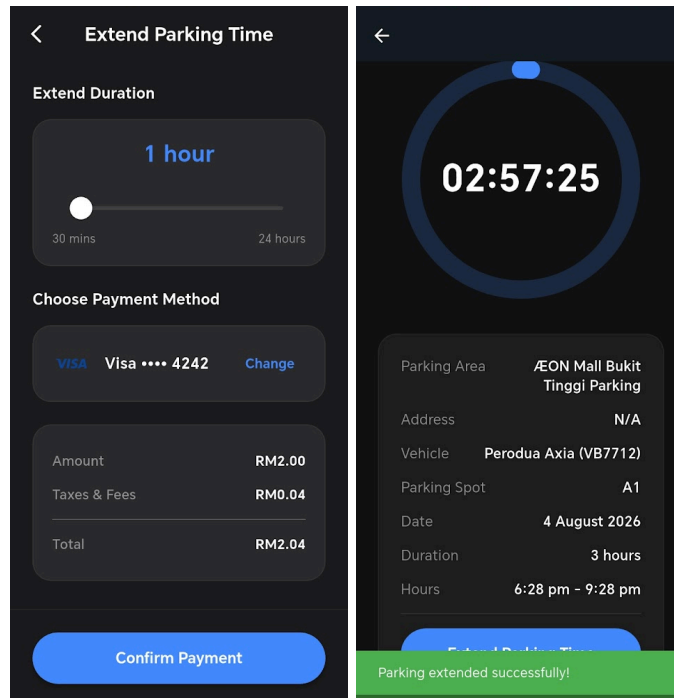


Figure 4.9: Extend Parking and Successful Page

Figure 4.9 displays the parking time extension flow and booking confirmation screen. Drivers can extend their current active parking session remotely by adding extra hours and processing additional charges without returning to their vehicle.

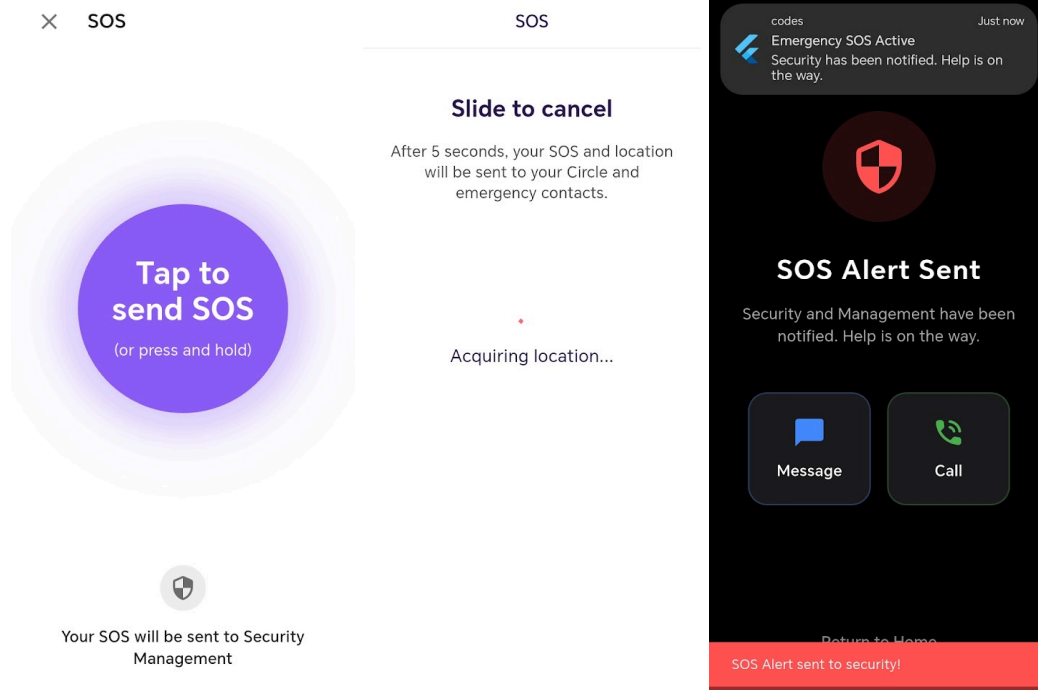


Figure 4.10: SOS / Emergency Panic Alert Trigger Overlay

Figure 4.10 presents the in-app SOS emergency panic trigger overlay. Tapping the emergency panic button immediately dispatches an urgent distress event to Cloud Firestore, sending live user location and contact details to the Web Admin Dashboard to trigger audio-visual alarms.

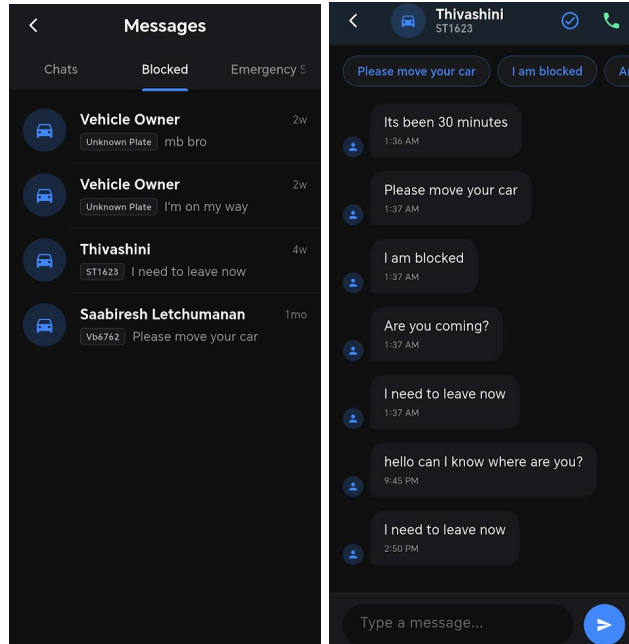


Figure 4.11: In-App Real-time Chat

Figure 4.11 shows the peer-to-peer driver messaging interface. This module enables direct real-time messaging between vehicle owners to coordinate vehicle movement during double-parking scenarios without exposing personal telephone numbers.

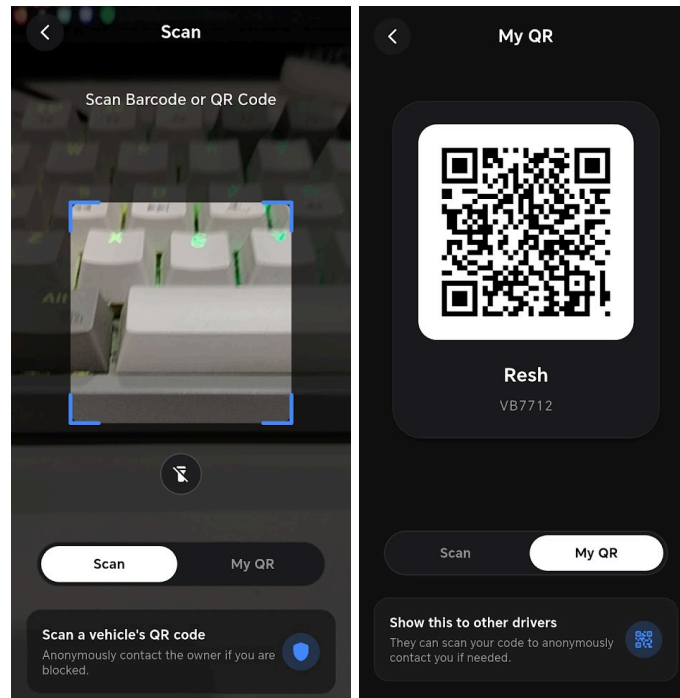


Figure 4.12: QR Code Scanner and Display for Vehicle Identification / Chat

Figure 4.12 illustrates the vehicle QR code display and camera scanner interface. Vehicle owners can display their unique encrypted windshield QR code, while other drivers can use the in-app scanner to initiate anonymous owner contact.

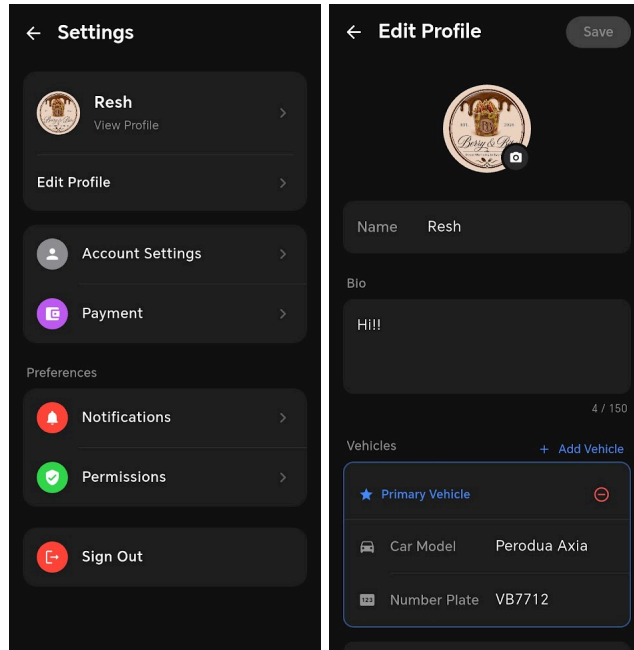


Figure 4.13: Main Settings & Edit Profile / Vehicle Page

Figure 4.13 presents the profile customization and multi-vehicle management screen. Users can update their personal details, upload a profile photo via local gallery picking, and manage multiple registered vehicles (make, model, and plate number).

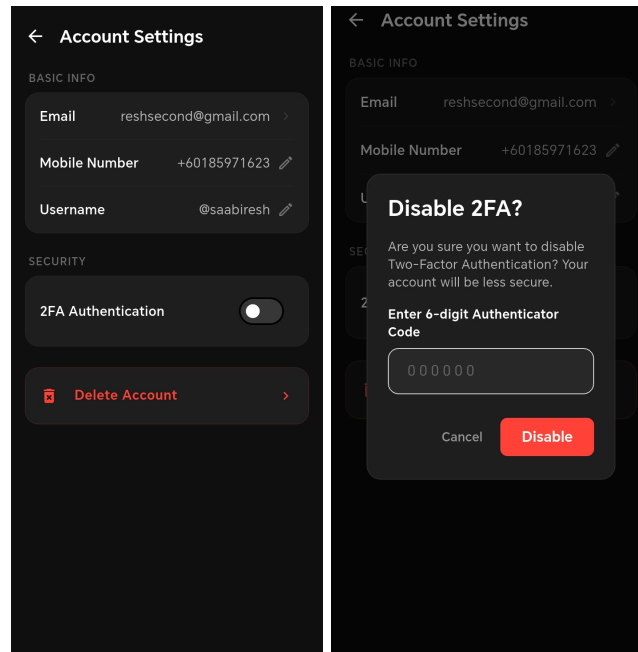


Figure 4.14: Account Settings and Two-Factor Authentication (2FA)

Figure 4.14 shows the security configuration panel and Two-Factor Authentication (2FA) setup. Users can enable TOTP-based 2FA by scanning a generated QR code using an authenticator app (such as Google Authenticator) to verify 6-digit security codes.

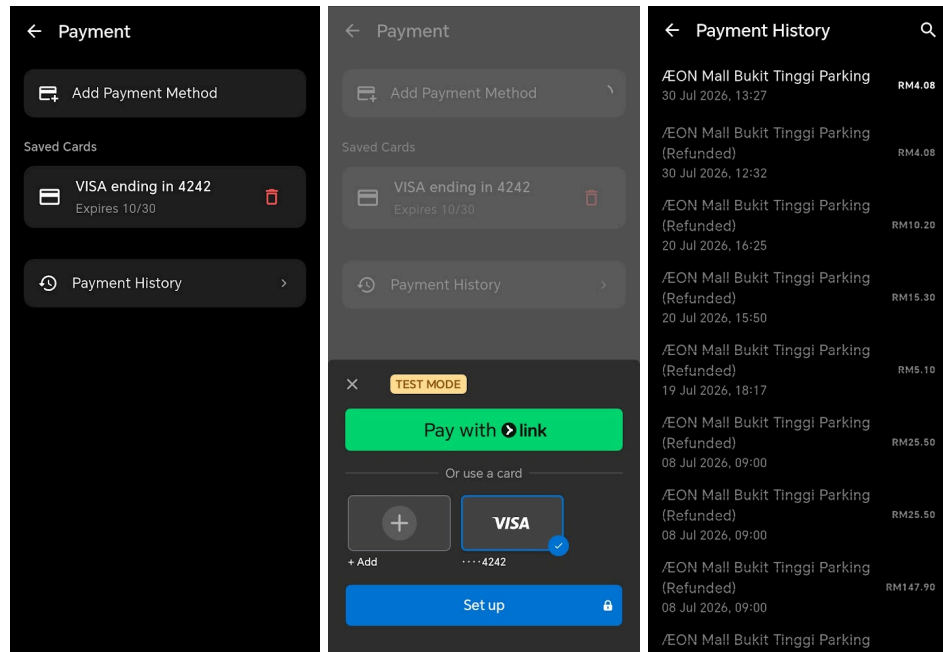


Figure 4.15: Payment Settings / Stripe Payment Method and Payment History

Figure 4.15 depicts saved payment methods and transaction history ledgers. Users can securely add or remove credit/debit card details managed by Stripe and review past receipts and booking payments.

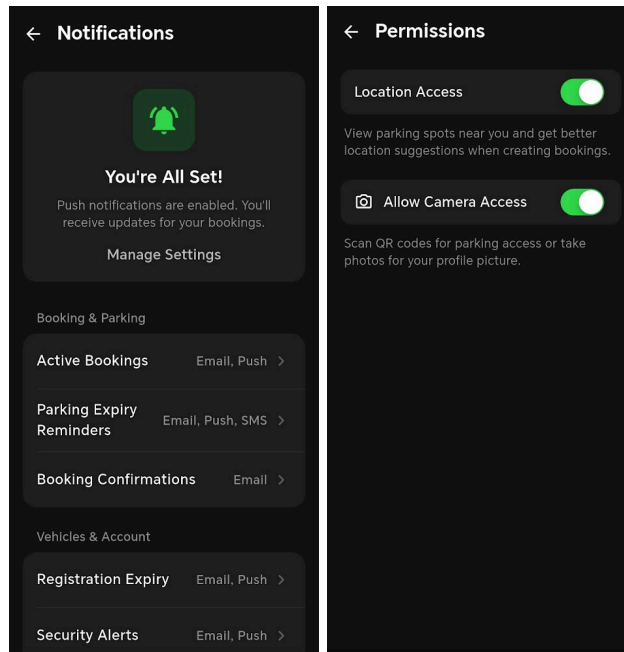


Figure 4.16: Push Notification Settings and Permissions Access Settings

Figure 4.16 illustrates the system access control and push notification preference panel. Drivers can toggle permissions for device location (for finding nearby facilities) and camera access (for QR scanning), as well as configure alerts for booking expiry and security notices.

Flutter Code Snippets (Frontend)

```
void _listenToRealtimeSensors() {
  final dbRef = FirebaseDatabase.instance.ref(_rtdbPath);
  _rtdbSubscription = dbRef.onValue.listen((DatabaseEvent event) {
    if (!mounted) return;

    final data = event.snapshot.value;
    if (data is Map) {
      setState(() {
        data.forEach((key, value) {
          final uiSlot = _slotMapping[key.toString()];
          if (uiSlot != null) {
            if (value.toString() == 'occupied') {
              _sensorOccupiedSpots.add(uiSlot);
              if (_selectedSpot == uiSlot) {
                _selectedSpot = null; // Deselect if occupied by sensor
              }
            } else if (value.toString() == 'available') {
              _sensorOccupiedSpots.remove(uiSlot);
            }
          }
        });
      });
    }
  });
}
```

Table 4.1: Real-time RTDB Sensor Status Stream Listener

This snippet demonstrates the stream listener that binds the Firebase Realtime Database (RTDB) hardware sensor states directly to the Flutter UI, dynamically altering slot availability.

```

@override
void initState() {
  super.initState();
  // Generate secure secret key on initialization
  _secret = TOTP.generateSecretKey();
}
@override
void didChangeDependencies() {
  super.didChangeDependencies();
  final authProvider = Provider.of<AuthProvider>(context, listen: false);
  _qrUri = TOTP.getOtpAuthUrl(secret: _secret, email: authProvider.email);
}

```

Table 4.2: Two-Factor Authentication (2FA) Setup & Secret Generation

Shows how the secret key is securely generated on screen initialization and mapped to standard TOTP `otpauth://` URI schema for compatibility with Google Authenticator.

```

final callable =
FirebaseFunctions.instance.httpsCallable('createSetupIntent');
final response = await callable.call();
final clientSecret = response.data['clientSecret'];
final customerId = response.data['customer'];
final ephemeralKey = response.data['ephemeralKey'];
if (clientSecret == null) {
  throw Exception('Failed to get setup intent secret');
}
await Stripe.instance.initPaymentSheet(
  paymentSheetParameters: SetupPaymentSheetParameters(
    setupIntentClientSecret: clientSecret,
    customerId: customerId,
    customerEphemeralKeySecret: ephemeralKey,
    merchantDisplayName: 'myMove',
    style: ThemeMode.dark,
  ),
);
await Stripe.instance.presentPaymentSheet();

```

Table 4.3: Stripe SDK Setup Intent Card Addition Integration

Secure client-side Stripe Payment Sheet integration. It triggers a secure Firebase HTTPS Callable Cloud Function to retrieve Stripe Setup Intent credentials (clientSecret, customer, ephemeralKey) and initializes/displays the Stripe SDK modal checkout sheet.


```

StreamBuilder<QuerySnapshot>(
  stream: FirebaseFirestore.instance
    .collection('bookings')
    .where('userId', isEqualTo: user.uid)
    .snapshots(),
  builder: (context, snapshot) {
    // ... validation and loading checks ...
    final allPayments = snapshot.data!.docs.where((doc) {
      final data = doc.data() as Map<String, dynamic>;
      final docStatus = data['status'] as String? ?? 'pending';
      return docStatus == 'completed' || docStatus == 'active' || docStatus
== 'canceled';
    }).map((doc) {
      final data = doc.data() as Map<String, dynamic>;
      final isLegacyPrice = !data.containsKey('totalPaid');
      final double basePrice = (data['totalPrice'] ?? 0).toDouble();
      final double priceAmount = isLegacyPrice
        ? basePrice * 1.02
        : (data['totalPaid'] ?? 0).toDouble();

      final startStr = data['startDateTime'] as String?;
      final date = startStr != null
        ? DateFormat('dd MMM yyyy,
HH:mm').format(DateTime.parse(startStr).toLocal())
        : 'Unknown Date';
      // ... maps to payment list elements ...
    });
  }
)

```

Table 4.4: Querying User Booking Transactions from Cloud Firestore

Demonstrates the real-time StreamBuilder query that pulls user-specific bookings from Cloud Firestore and dynamically maps them to a transaction history model.

```

Future<void> _handlePermissionRequest(Permission permission, bool
isSwitchEnabled) async {
    final status = await permission.status;

    if (status.isPermanentlyDenied || (!isSwitchEnabled && status.isGranted))
    {
        // If permanently denied, or user wants to revoke, they must go to
settings
        await openAppSettings();
    } else if (isSwitchEnabled && !status.isGranted) {
        final newStatus = await permission.request();
        if (newStatus.isPermanentlyDenied) {
            await openAppSettings();
        }
    }
    await _checkPermissions();
}

```

Table 4.5: Dynamic System Permissions Handler

Coordinates OS-level permission requests using the `permission_handler` package, handling standard prompts and permanent denials by redirecting users to system settings as needed.

- **Part 2: Web Scanner (Guest Request Portal)**

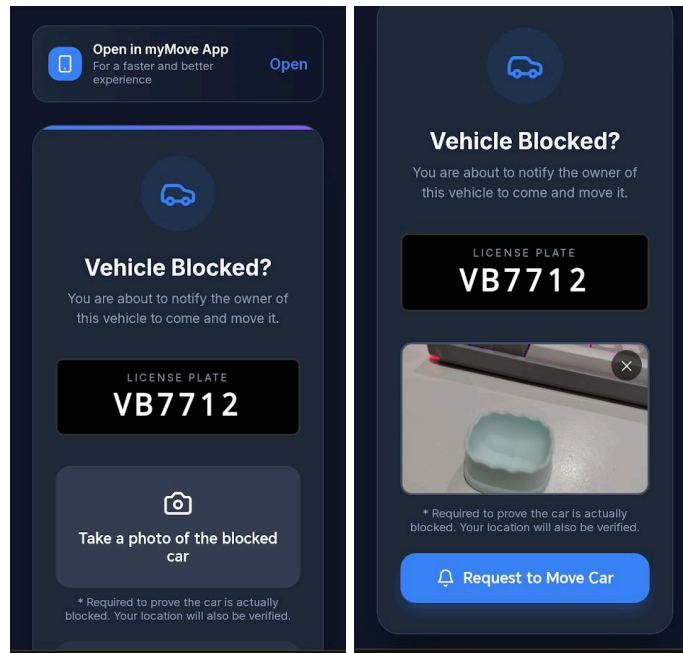


Figure 4.17: Webpage for Blocked Vehicle QR Scan and Photo Proof Verification

Figure 4.17 displays the lightweight, zero-install Scanner-Web guest portal loaded in a mobile browser upon scanning a vehicle's windshield QR code. Guest drivers can capture and upload photo proof of a double-parking blockage and submit a move request anonymously.

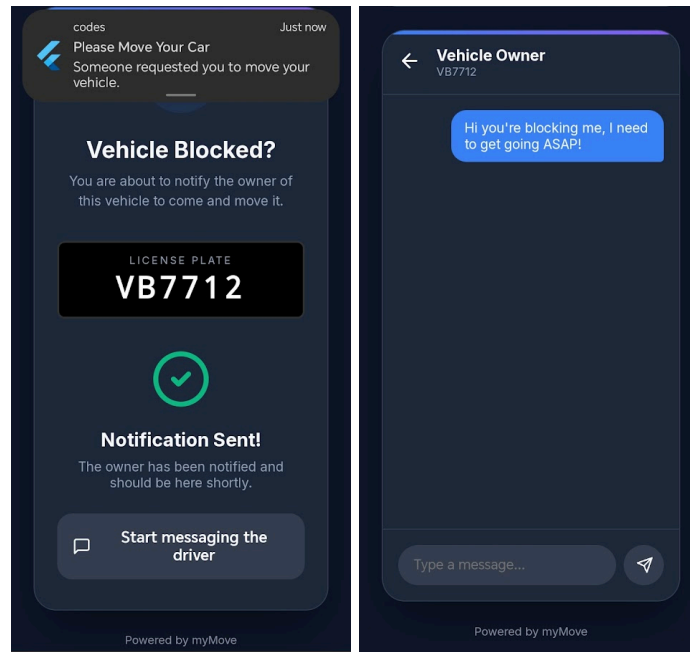


Figure 4.18: Successful Request Notification and Web-Based Chat

Figure 4.18 shows the notification confirmation screen and guest web messaging view. Once submitted, the system dispatches an instant FCM push notification to the vehicle owner while establishing an anonymous temporary web chat session.

Web Scanner Code Snippets (Frontend - Typescript)

```
const handleRequestMove = async () => {
  if (!targetId || requestStatus === 'sending' || !user || !photoFile)
    return;
  setRequestStatus('sending');
  try {
    // 1. Fetch Guest Geolocation
    const location = await getLocation();

    // 2. Upload verification photo to Cloud Storage
    const photoRef = ref(storage,
`move_requests/${user.uid}_${Date.now()}.jpg`);
    const metadata = { contentType: photoFile.type || 'image/jpeg' };
    await uploadBytes(photoRef, photoFile, metadata);
    const photoUrl = await getDownloadURL(photoRef);

    // 3. Trigger Firebase HTTPS Callable Cloud Function
    const requestWebMoveCar = httpsCallable(functions, 'requestWebMoveCar');
    await requestWebMoveCar({
      targetUserId: targetId,
      photoUrl,
      location
    });
    setRequestStatus('success');
  } catch (err: any) {
    console.error('Error requesting move:', err);
    setRequestStatus('error');
    setError(err.code === 'functions/resource-exhausted'
      ? 'Please wait a moment before sending another notification to prevent
spam.'
      : 'Failed to send notification.');
```

Table 4.6: Guest Move Request Submission Logic

Shows the client-side flow: validating parameters, fetching geolocated coordinates, uploading the image verification to Firebase Storage, and executing the secure Cloud Function.

- **Part 3: Admin Web Dashboard (Vite + React)**

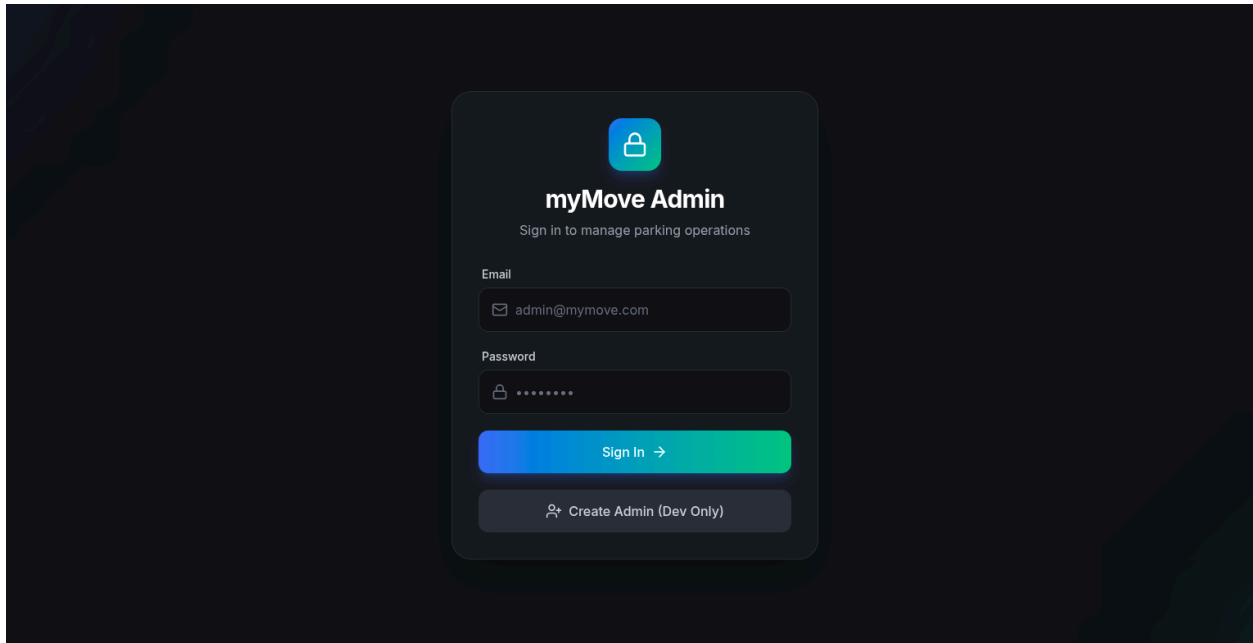


Figure 4.19: Admin Panel Login

Figure 4.19 presents the secure administrative web portal login screen. Facility managers and security staff authenticate with admin credentials to gain role-based access to management controls.

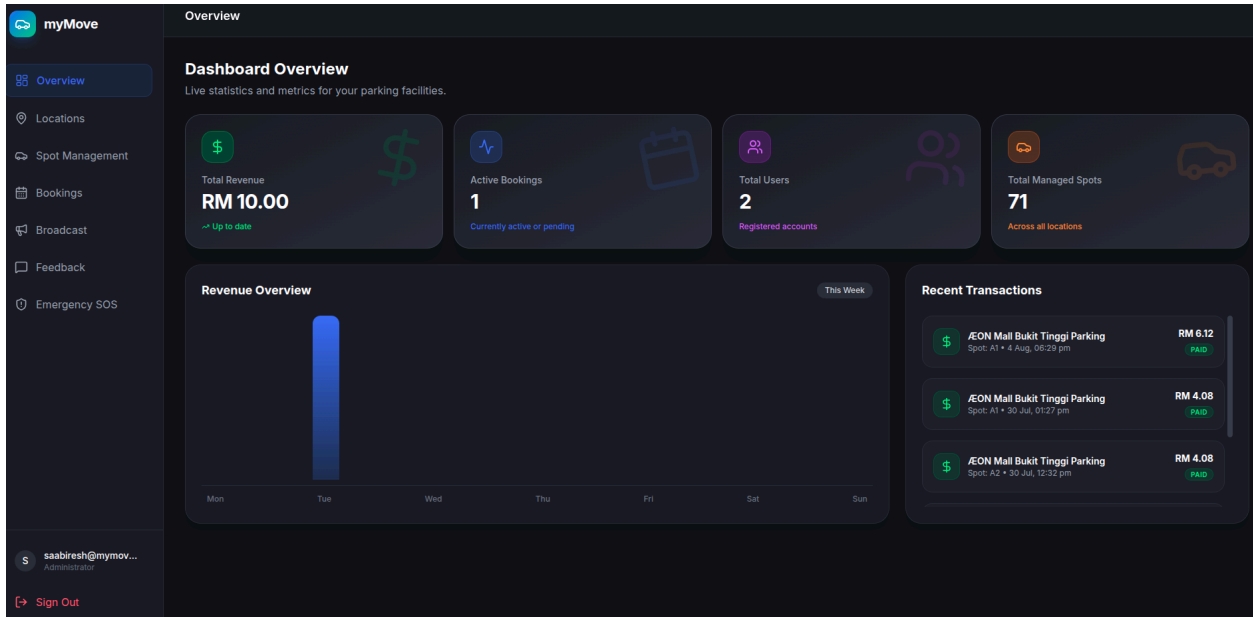


Figure 4.20: Admin Dashboard with Analytics and Weekly Revenue

Figure 4.20 shows the main admin dashboard featuring real-time occupancy statistics, facility performance metrics, active emergency alerts, and visual weekly revenue charts.

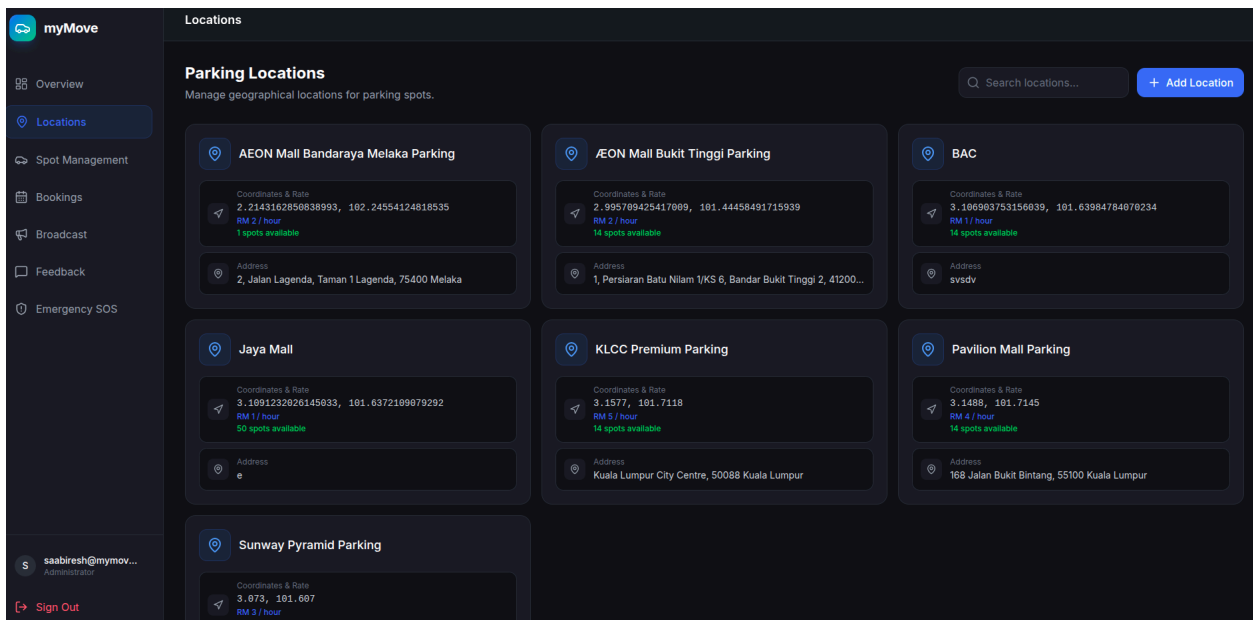


Figure 4.21: Parking Location Management Panel

Figure 4.21 shows the parking location management interface. Administrators can create, edit, or remove parking structures (CRUD operations), configure total slot capacities, and set hourly rates.

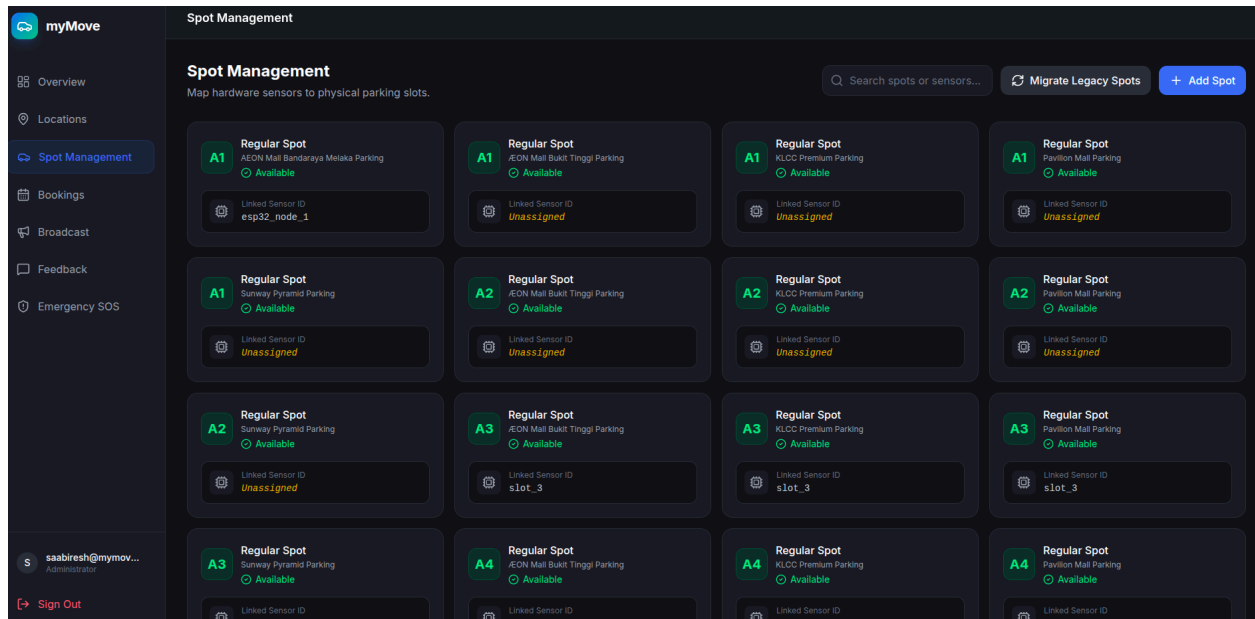


Figure 4.22: Manage Parking Spot

Figure 4.22 shows individual parking spot mapping and active booking monitors. Administrators can manually inspect slot occupancy statuses, pair hardware sensor IDs to physical bays, and oversee active reservation lists.

LOCATION & SPOT	DATE & TIME	VEHICLE	AMOUNT	STATUS	ACTIONS
AEON Mall Bukit Tinggi Parking Spot: A1	4 Aug 2026, 06:28 pm to: 4 Aug 2026, 09:28 pm	VB7712 Periodua Axia	RM 6.12	ACTIVE	
AEON Mall Bukit Tinggi Parking Spot: A1	30 Jul 2026, 01:27 pm to: 30 Jul 2026, 03:27 pm	VB7712 Periodua Axia	RM 4.08	COMPLETED	
AEON Mall Bukit Tinggi Parking Spot: A2	30 Jul 2026, 12:32 pm to: 30 Jul 2026, 02:32 pm	VB7712 Periodua Axia	RM 4.08	CANCELED	
AEON Mall Bukit Tinggi Parking Spot: A1	20 Jul 2026, 04:25 pm to: 20 Jul 2026, 06:25 pm	VB7712 Periodua Axia	RM 10.20	CANCELED	
AEON Mall Bukit Tinggi Parking Spot: A1	20 Jul 2026, 03:50 pm to: 20 Jul 2026, 06:00 pm	VB7712 Periodua Axia	RM 15.30	CANCELED	
AEON Mall Bukit Tinggi Parking Spot: A1	19 Jul 2026, 06:17 pm to: 19 Jul 2026, 06:30 pm	VB7712 Periodua Axia	RM 5.10	CANCELED	
AEON Mall Bukit Tinggi Parking Spot: A2	8 Jul 2026, 09:00 am to: 9 Jul 2026, 02:00 pm	VB7712 Periodua Axia	RM 147.90	CANCELED	
AEON Mall Bukit Tinggi Parking Spot: A1	8 Jul 2026, 09:00 am to: 8 Jul 2026, 02:00 pm	VB7712 Periodua Axia	RM 25.50	CANCELED	
AEON Mall Bukit Tinggi Parking Spot: A1	8 Jul 2026, 09:00 am to: 8 Jul 2026, 02:00 pm	VB7712 Periodua Axia	RM 25.50	CANCELED	

Figure 4.23: Booking Management Monitor

Figure 4.23 illustrates the real-time booking management monitor on the Web Admin Dashboard. Administrators can view, filter, and track all active, upcoming, and completed user reservations across facilities, monitor live payment statuses processed via Stripe, and inspect detailed user booking logs.

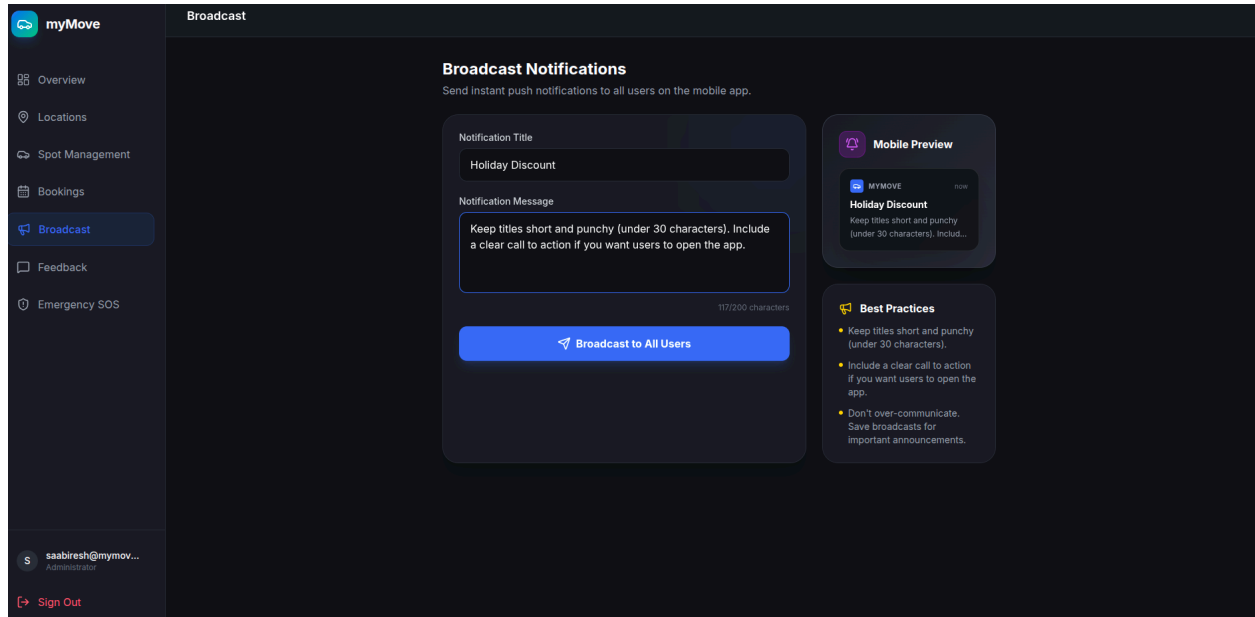


Figure 4.24: Broadcast Notification to All Users

Figure 4.24 illustrates the system-wide announcement broadcast panel. Administrators can compose and send push notifications to all registered mobile app users regarding maintenance schedules or urgent security notices.

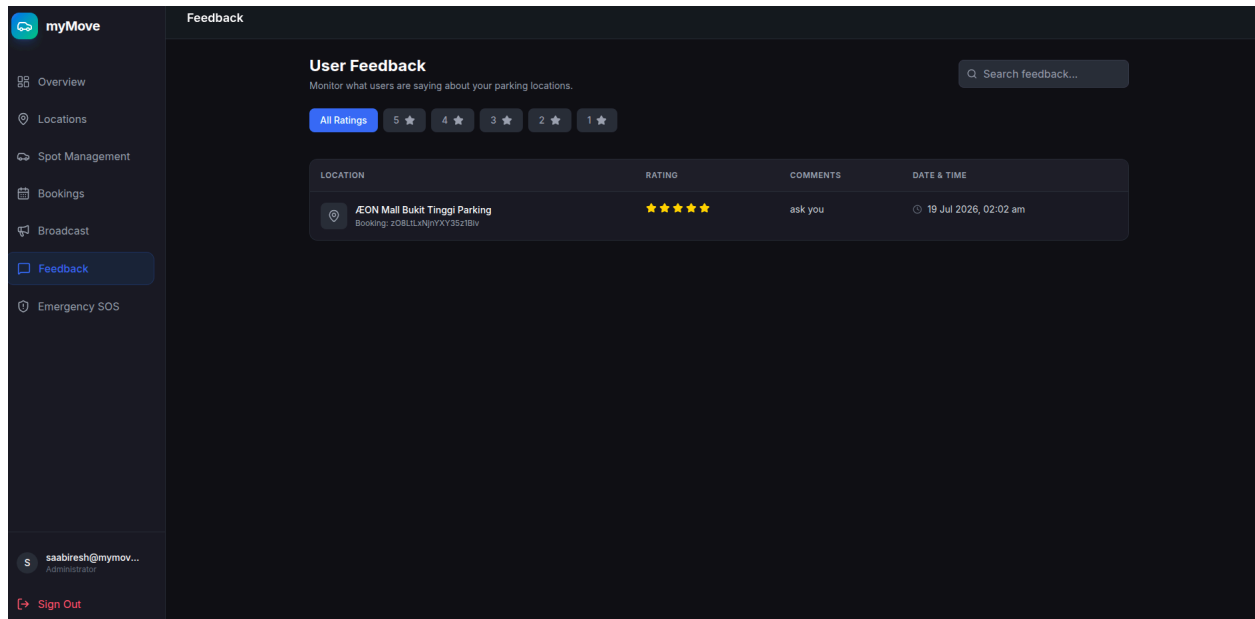


Figure 4.25: User Acceptance Testing Feedback Page

Figure 4.25 shows the User Acceptance Testing (UAT) feedback aggregation panel on the admin dashboard. This page compiles user-submitted survey responses, rating distributions, and qualitative feedback to evaluate system usability, driver satisfaction, and feature effectiveness.

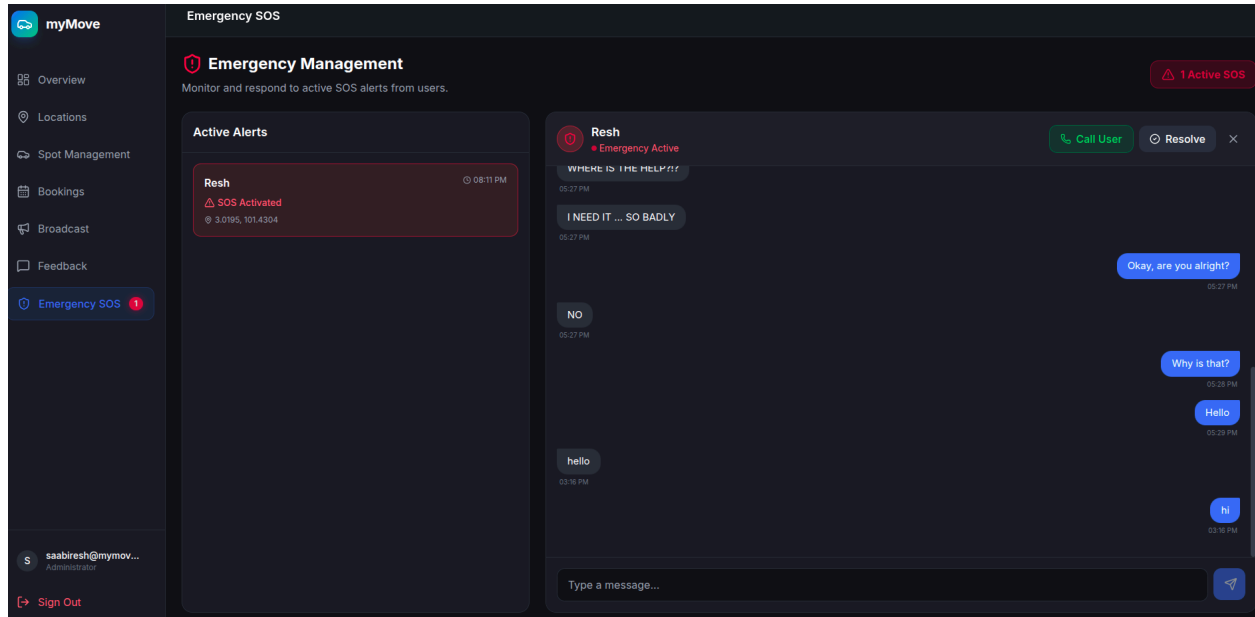


Figure 4.26: SOS / Emergency Panic Trigger Alarm

Figure 4.26 displays the live SOS emergency alert screen on the admin dashboard. When a driver triggers an in-app emergency alert, the dashboard immediately triggers an audible alarm siren and displays a pop-up modal showing the user's vehicle details, contact information, and exact GPS location on an interactive map for rapid security response.

- **Real-time Active Booking Calculations & Listeners**

```
bookings.forEach(booking => {
  if (booking.status !== 'canceled') {
    totalRevenue += (booking.totalPrice || booking.totalPaid || 0);
  }
  // Real-time effective active status check
  const now = new Date();
  const endDateTime = booking.endDateTime ? new Date(booking.endDateTime) :
null;
  let isEffectiveActive = false;
  if (booking.status === 'pending') {
    isEffectiveActive = true;
  } else if (booking.status === 'active') {
    if (!endDateTime || endDateTime >= now) {
      isEffectiveActive = true;
    }
  }
  if (isEffectiveActive) {
    activeBookings++;
  }
});
```

Table 4.7: Real-Time Active Booking Calculations & Listeners

Aggregates booking collections in real time, validating "Active" states on-the-fly by checking time boundaries (**endDateTime**) against local device time.

4.5 Backend Implementation

4.5.1 Backend Architecture

myMove uses a serverless Backend-as-a-Service (BaaS) architecture powered by Google Firebase. This means there is no need to manage a dedicated physical or virtual server. Instead, the Flutter mobile app and the React/Vite admin web portal connect directly and securely to Firebase's cloud services using authenticated SDKs.

This architecture ensures that real-time sensor updates happen in less than a second, makes user login and account management easier, and can automatically handle hundreds of drivers and administrators at the same time.

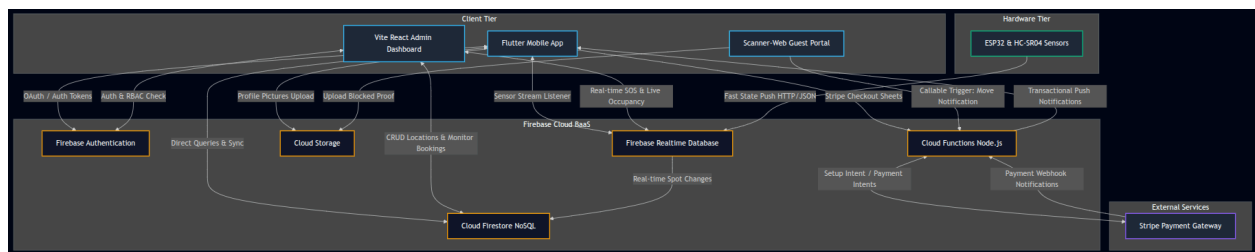


Figure 4.27: myMove BaaS Architecture Diagram

The system is divided into four main components:

1. Client Tier:

- **Flutter Mobile App:** Written in Dart, serving as the user interface for drivers. It accesses Firebase services via the official **firebase_core**, **firebase_auth**, **cloud_firestore**, **firebase_database**, and **firebase_messaging** plugins.
- **React Web Admin Dashboard:** A responsive administration tool built with React and Vite. It allows managers to audit reports, view revenue analytics, manage locations, and monitor active SOS situations.
- **Scanner-Web Portal:** A lightweight React-based web interface designed for guests. When a guest scans the QR code affixed to a vehicle blocking their path, this site opens instantly without requiring an app store download.

2. **Firebase Cloud BaaS:**

- **Authentication:** Manages registration and sign-in using secure JSON Web Tokens (JWT). It supports standard email/password authentication, linking physical phone numbers, and coordinates session persistence.
- **Cloud Firestore:** A flexible, document-oriented NoSQL database that houses all persistent relational data (e.g., profiles, booking ledgers, active chat sessions, locations).
- **Firebase Realtime Database (RTDB):** A low-latency, WebSocket-based JSON database. It is optimized to stream the binary state (**available** or **occupied**) of physical parking spots from hardware microcontrollers straight to connected clients within milliseconds.
- **Cloud Storage:** Serves as a secure object store for binary assets, handling driver verification photos for blocked move requests and user-uploaded profile pictures.
- **Cloud Functions:** A serverless Node.js backend executing secure operations that cannot be trusted to client code (e.g., Stripe Payment Intent signing, rate-limiting guest requests, and validating active booking overlaps).

3. **Hardware Tier (IoT System):**

- **ESP32 Microcontroller:** Wired to ultrasonic distance sensors (HC-SR04) monitoring physical slots. It runs firmware that computes parking slot occupancy and pushes status changes directly to the Firebase Realtime Database path.

4. **External Infrastructure:**

- **Stripe Payment Gateway:** Handles payment card entry and processing. The client requests a secure session through Cloud Functions, which returns a setup intent token allowing the Stripe iOS/Android SDK to securely present a payment form, eliminating local database compliance risks.

4.6 Database Design

The database uses a design that combines Cloud Firestore and Firebase Realtime Database (RTDB). Cloud Firestore stores structured and permanent data, while Firebase Realtime Database (RTDB) handles real-time parking sensor updates. Since Firestore is a document-based database, data is stored in collections and documents instead of tables and rows. Relationships between different collections are created by storing unique IDs inside documents. Data security is managed using Firestore Security Rules. These rules ensure that only authorized users can access or modify sensitive information, helping to protect the system's data.

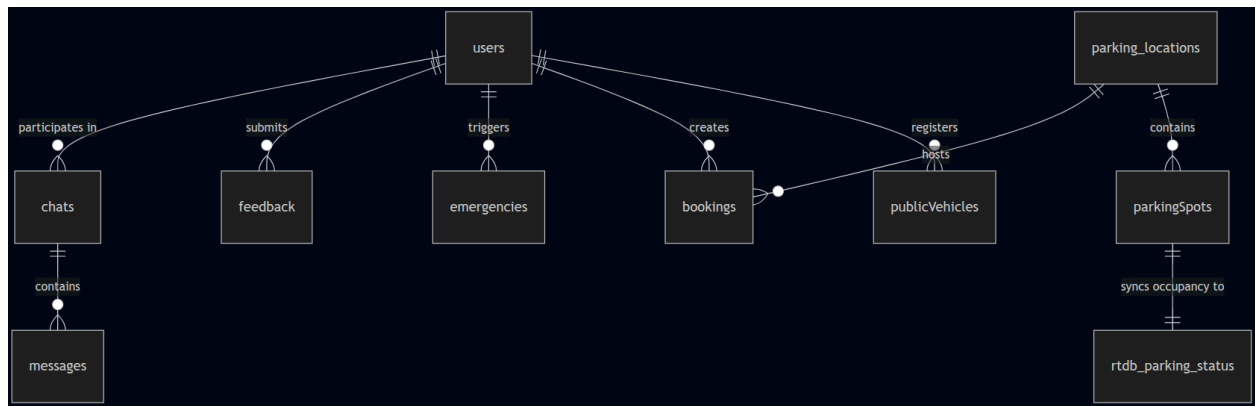


Figure 4.28: ER Diagram of Cloud Firestore Database Schema

4.6.1 Cloud Firestore Collections Schema

The database contains the following primary collections:

<u>Field Name</u>	<u>Data Type</u>	<u>Description</u>
uid (Document ID)	String	Unique identifier linked to Firebase Auth.
displayName	String	User's full name
email	String	System access tier: user or admin
twoFactorSecret	String	Base32 secret key used for Google Authenticator TOTP authentication.
isTwoFactorEnabled	Boolean	True if Two-Factor Authentication is active.
fcmToken	String	Firebase Cloud Messaging token for push notifications.
vehicles	Array (Map)	Nested list of user vehicles: [{ brand , model , plateNumber , isPrimary }].
createdAt	Timestamp	Timestamp indicating account registration date.

Table 4.8: Cloud Firestore users Collection Schema

Stores user profiles, roles, notification tokens, and security keys needed for Multi-Factor Authentication (MFA).

<u>Field Name</u>	<u>Data Type</u>	<u>Description</u>
vehicleId (Document ID)	String	Matches the registered owner's UID
ownerID	String	References the users UID
plateNumber	String	License plate number (indexed for fast search)
brand	String	Vehicle manufacturer brand.
model	String	Specific vehicle model name.
color	String	Vehicle color description
isActive	Boolean	Active flag indicating if the vehicle is currently in use.
createdAt	Timestamp	Registration timestamp

Table 4.9: Cloud Firestore publicVehicles Collection Schema

A vehicle registry that stores license plates and vehicle brands with owner IDs to enable fast QR code lookup.

<u>Field Name</u>	<u>Data Type</u>	<u>Description</u>
locationId (Document ID)	String	Unique parking location identifier.
name	String	Human-readable building name (e.g. Main Block)
address	String	Full physical street address.
latitude	String	GPS latitude coordinate
longitude	String	GPS longitude coordinate
hourlyRate	String	Cost per hour of parking (in MYR)
operatingHours	Boolean	Opening and closing rules (e.g. "08:00 - 22:00")

Table 4.10: Cloud Firestore parking_locations Collection Schema

Contains structural information and pricing rates for active parking facility buildings.

<u>Field Name</u>	<u>Data Type</u>	<u>Description</u>
spotId (Document ID)	String	Composite key combining location and spot name.
locationId	String	Foreign Key referencing parking_locations.locationId
spotName	String	User display code (e.g., "A1", "A2").
hardwareSensorId	String	Unique hardware sensor ID printed on the physical sensor.
createdAt	Timestamp	Creation date of mapping.

Table 4.11: Cloud Firestore parkingSpots Collection Schema

Maps user-facing layout spots to physical hardware sensor units.

<u>Field Name</u>	<u>Data Type</u>	<u>Description</u>
bookingId (Document ID)	String	A unique and secure ID created for each transaction.
userId	String	Foreign Key referencing the driver's users.uid
locationId	String	Foreign Key referencing parking_locations.locationId .
spotName	String	The reserved parking spot code.
startDateTime	String (ISO)	Reservation start boundary.
endDateTime	String (ISO)	Reservation expiration boundary.
status	String	Booking status: pending, active, completed, canceled
totalPrice	Number	Initial calculated rate.
totalPaid	Number	Final actual payment captured (with Stripe processing fees).
paymentIntentId	String	Reference ID mapping to the payment in the Stripe Dashboard.
createdAt	Timestamp	Creation timestamp.

Table 4.12: Cloud Firestore bookings Collection Schema

Stores parking reservation records, including booking duration, payment details, and status updates..

<u>Field Name</u>	<u>Data Type</u>	<u>Description</u>
chatId (Document ID)	String	A unique and secure ID created for each transaction.
participants	String	Foreign Key referencing the driver's users.uid
lastMessage	String	Foreign Key referencing parking_locations.locationId .
lastMessageAt	String	The reserved parking spot code.
blockedDriverId	String (ISO)	Reservation start boundary.
vehicleOwnerId	String (ISO)	Reservation expiration boundary.

Table 4.13: Cloud Firestore chats & messages (Subcollection) Schema

Stores active real-time chat sessions and peer-to-peer messages between drivers during double-parking coordination.

<u>Field Name</u>	<u>Data Type</u>	<u>Description</u>
messageId (Document ID)	String	Unique message ID
senderId	String	UID of the message sender
receiverId	String	UID of the message receiver.
messageText	String	Decrypted message content
isRead	Boolean	True if the receiver has viewed the message
createdAt	Timestamp	Timestamp representing exact delivery time

Table 4.14: Cloud Firestore chats Collection & messages Subcollection Schema

<u>Field Name</u>	<u>Data Type</u>	<u>Description</u>
emergencyId (Document ID)	String	Unique alert ID
userId	String	Foreign Key referencing the reporting users.uid .
locationAddress	String	Nearest street name fetched via geolocation APIs.
latitude	Number	Exact trigger latitude
longitude	Number	Exact trigger longitude
status	String	Management status: active or resolved
createdAt	Timestamp	Timestamp of alert trigger

Table 4.15: Cloud Firestore emergencies Collection Schema

Central emergency dashboard queue tracking panic alerts triggered by mobile users.

<u>Field Name</u>	<u>Data Type</u>	<u>Description</u>
feedbackId (Document ID)	String	Unique feedback ID
userId	String	Foreign Key referencing users.uid
score	Number	Satisfaction rating from 1 to 5
comments	String	Open-ended user remarks
createdAt	Timestamp	Submission timestamp

Table 4.16: Cloud Firestore feedback Collection Schema

Collects user acceptance feedback scores and textual evaluations during testing phases.

4.6.2 Firebase Realtime Database (RTDB) Schema

The Realtime Database uses a schema-less JSON structure designed for fast read and write operations by low-power IoT microcontrollers. Data is organized using location and parking spot keys to minimize data transfer size and improve efficiency.

```
{
  "parking_status": {
    "location_A": {
      "slot_1": "occupied",
      "slot_2": "available",
      "slot_3": "occupied"
    },
    "location_B": {
      "slot_1": "available",
      "slot_2": "available"
    }
  }
}
```

Table 4.17: Firebase Realtime Database (RTDB) Schema

- Data Path: **/parking_status/{locationId}/{spotName}**
- Payload values:
 - **"available"**: Ultrasonic distance measurement exceeds the threshold (> 10.0 cm), indicating the spot is vacant.
 - **"occupied"**: Distance measurement is within the threshold (≤ 10.0 cm), indicating a vehicle is present.

4.7 Summary

The implementation phase successfully converted the system requirements and design into a fully functional, secure, and integrated IoT-based smart parking system. By using Flutter for the cross-platform mobile application, ESP32 microcontrollers with HC-SR04 ultrasonic sensors for real-time parking spot monitoring, and Firebase services (Cloud Firestore and Realtime Database) as the backend, myMove provides live parking updates with reliable data handling. The integration of the Stripe SDK enables secure payment processing, while security features such as Two-Factor Authentication (2FA) and QR-based contact verification improve system safety and user convenience. Firebase Cloud Functions manage the serverless business logic, while the React/Vite Web Admin Dashboard allows administrators to monitor and manage the system efficiently. Overall, the completed myMove application meets the project objectives by providing a scalable, maintainable, and improved solution compared to traditional parking management and vehicle blocking methods.

CHAPTER 5

TESTING & EVALUATION

5.1 Introduction

This chapter presents the evaluation, testing outcomes, and user acceptance results of the myMove: IoT Enabled Smart Parking Reservation and Privacy Vehicle Communication System. Aligning with the rigorous validation phase of the engineering methodology, the system underwent comprehensive testing across multiple tiers: unit testing, integration testing, system testing, and User Acceptance Testing (UAT). The primary objectives of these evaluations are to verify that the integrated IoT hardware, the mobile application, the guest web portal, and the serverless backend operate reliably, securely, and within acceptable latency parameters. Testing was conducted on both physical smartphones (Android device) and emulation environments to evaluate cross-platform consistency and system resilience under typical operational loads. The results detailed in this chapter confirm that the myMove system successfully achieves all functional and performance requirements outlined in the system specification.

5.2 Project Deliverables

The development phase produced a complete set of software and hardware components that work together to form the myMove:

1. An Android mobile app with user login, real-time parking availability, booking and Stripe payments, profile and vehicle management, 2FA security, permissions, and driver chat.
2. A responsive web portal developed with React, TypeScript, and Vite, designed for parking lot operators. It provides location and slot configuration (CRUD), live occupancy monitoring, emergency/SOS alert routing, and visual revenue charts.
3. A lightweight web portal designed for mobile browsers, loaded instantly when a guest scans the QR code on a double-parked vehicle. It allows guests to notify vehicle owners of blocking situations anonymously without installing the mobile app.
4. A 3D physical prototype using an ESP32, three ultrasonic sensors, status and custom firmware for real-time parking updates.
5. A cloud-based backend using Firebase for user authentication, data storage, real-time sensor updates, payments, 2FA, and notifications.
6. A secure private repository structured cleanly into separate modules representing the mobile frontend, web admin, guest web scanner, Firebase Cloud Functions, and ESP32 microcontroller firmware.
7. Project diagrams, database designs, requirements documents, and setup guides on docs.

5.3 Software & Hardware Testing

Testing was conducted systematically to ensure the application is stable, secure, and userfriendly. Four main types of testing were performed: unit testing, integration testing, system testing, and usability testing.

5.3.1 Unit Testing

Unit tests were used to check individual functions, interface components, and system logic using Jest and Dart testing tools, along with manual boundary testing.

Key unit test categories included:

- **User Authentication:** Validating format rules for emails, checking password strength parameters, and verifying the TOTP generation and verification algorithms.
- **IoT Occupancy Algorithms:** Testing the distance measurement threshold logic where a distance of less than 10.0 cm is translated to "occupied" and anything above is marked "available".
- **Data Formatting & Validation:** Validating vehicle license plate inputs, formatting Stripe transaction amounts, and parsing raw Firestore timestamps.
- **URL Parameter Construction:** Verifying that the QR code generator correctly formats target Scanner-Web URLs containing encrypted vehicle and owner identifiers.

A total of 45 unit tests were executed, with a final pass rate of 97.8% and fixed the issue immediately. A summary of unit testing is detailed in Table 5.1:

Test Category	Number of Tests	Passed	Failed	Pass Rate
Authentication & 2FA	15	15	0	100%
IoT Distance Logic	10	10	0	100%
Input Form Validation	12	11	1	91.7%
QR Code & URL Utility	8	8	0	100%
Total	45	44	1	97.8%

Table 5.1: Unit Testing Summary

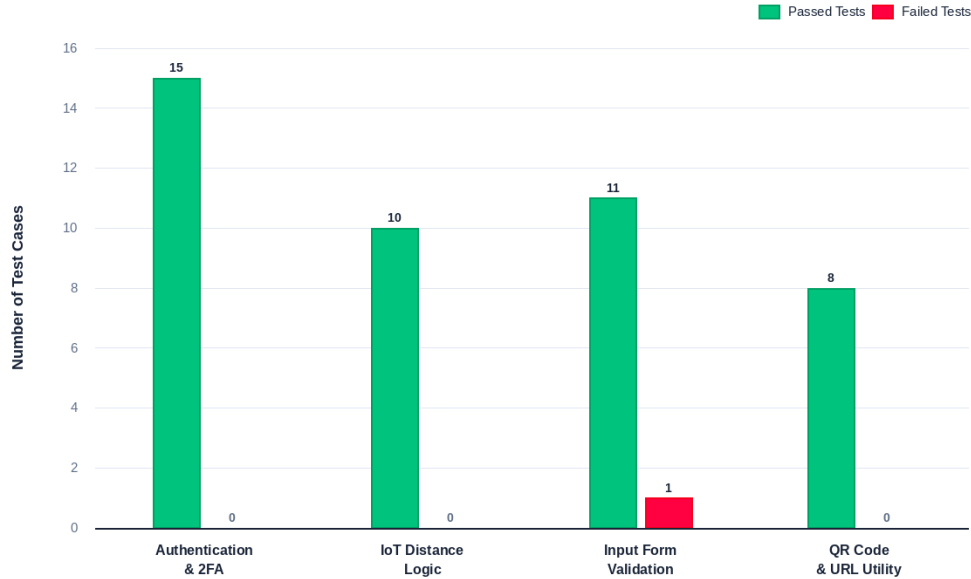


Figure 5.1: Unit Testing Execution Summary by Category

5.3.2 Integration Testing

Integration testing verified the communication links between separate system modules, focusing on the connection between the frontend clients, the physical IoT hardware, and the Firebase backend services.

Key integration scenarios tested:

1. **IoT Sensor-to-App Data Flow:** Verifying that the ESP32 successfully triggers an HTTP PUT request upon sensor state transition, updating the Firebase RTDB, which is immediately broadcast via reactive stream listener to update the Flutter UI color layout.
2. **Stripe Payment Subsystem:** Confirming that initiating a checkout sheet in Flutter triggers the backend Cloud Function to request a Setup Intent from Stripe, returning credentials to display the payment sheet and write card details securely to Firestore.
3. **Double-Parking QR Contact Flow:** Verifying that scanning a vehicle QR code redirects to the Scanner-Web page, and clicking "Notify Owner" calls the Firebase Cloud Function to dispatch an FCM push notification, triggering a visual alert in the owner's Flutter app.
4. **SOS Emergency Alert:** Verifying that tapping the SOS button in Flutter writes an emergency record to Firestore, which activates the live listener on the React Admin dashboard, triggering an immediate audio-visual alert modal.

Integration Path	Test Cases	Passed	Failed	Results
IoT ESP32 → RTDB → Flutter Grid	10	10	0	Spot color updates within 1.5 seconds.
Stripe Sheet → Cloud Functions → Firestore	8	8	0	Tokenization and card mapping complete successfully.
Web-Scanner → Cloud Functions → FCM → Mobile	12	12	0	The owner receives an anonymous alert successfully.
Flutter SOS button → Firestore → Web Admin	8	8	0	Alarm rings and shows an active location on the admin map.
Total	35	38	0	100%

Table 5.2: Integration Testing Summary

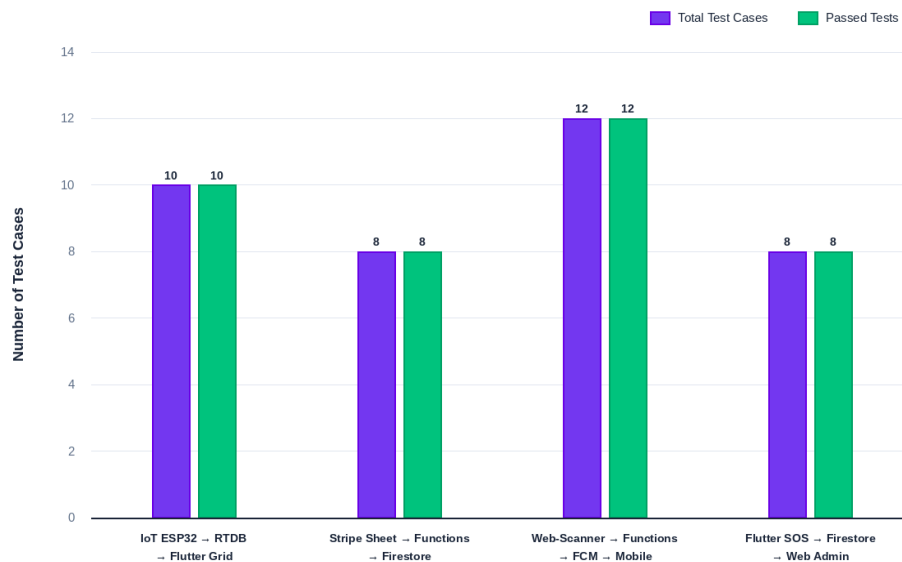


Figure 5.2: Integration Testing Execution Summary by Path

5.3.3 System Testing

System testing evaluated the complete, end-to-end user journeys under realistic operating conditions. The primary focus was measuring system response times, latency, and stability across different physical hardware.

The system tests focused on four critical paths:

1. **End-to-End Reservation & Payment Cycle:** Searching a facility, selecting an available spot, booking, paying via Stripe, and confirming slot reservation updates.
2. **Sensor Occupancy Sync Speed:** Measuring the time elapsed between placing an obstacle in front of the HC-SR04 sensor and the spot color changing to light-grey in the mobile app.
3. **Double-Parking Alert Delivery Latency:** Measuring the time between a guest clicking the "Notify Owner" button on a mobile web browser and the owner receiving the push notification.
4. **SOS Broadcast Speed:** Measuring the delay in displaying the emergency card on the React Admin dashboard after a user triggers an SOS event in-app.

System performance measurements are summarized in Table 5.3:

Test Scenario	Devices Tested	Success Rate	Response Time
Complete Reservation & Payment	Android, React Web	100%	4.2 seconds
IoT Spot Occupancy Update	ESP32, Firebase RTDB, Android	100%	1.1 seconds
Double-Parking QR Notification	Android (Browser)	100%	1.9 seconds
SOS Emergency Broadcast	Android (App), Browser (Admin Web)	100%	0.8 seconds

Table 5.3: System Testing & Performance Summary

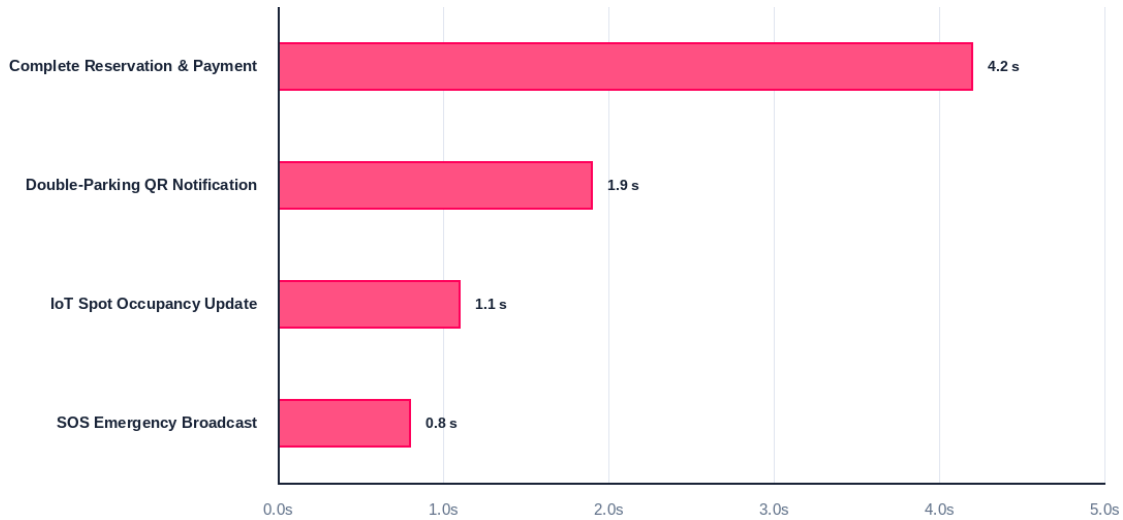


Figure 5.3: Response Time Graph in Seconds

5.3.4 User Acceptance Testing (UAT)

To evaluate usability and user satisfaction, UAT was conducted with 15 participants, including university students, staff, and daily commuters. As the IoT sensor system was only a tabletop prototype, participants tested the mobile application instead. The app was provided as an APK and tested on their personal Android smartphones.

UAT Methodology

Participants installed the myMove application on their Android devices using the provided APK file and were asked to complete a set of core software tasks to evaluate the app's functions and interface:

1. Register a new account and set up Two-Factor Authentication.
2. Add a vehicle make, model, and plate number, and view the generated QR code.
3. Browse parking locations, view rates, and book a simulated spot.
4. Scan a printed vehicle QR code using their phone's camera, access the anonymous Web-Scanner page, and send a double-parking notification.
5. Trigger an SOS alert and verify the backend response.

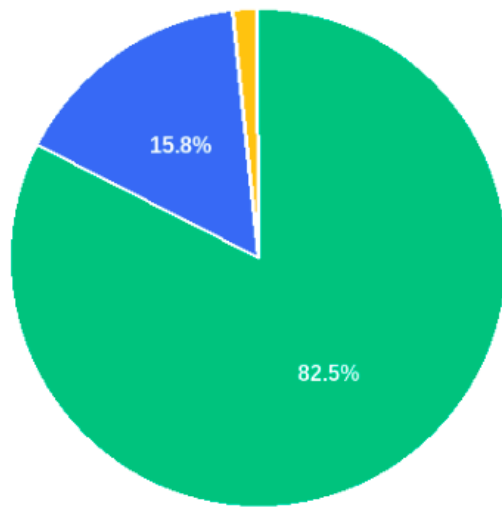
NOTE: A live demonstration of the ESP32 tabletop prototype was shown to the participants to illustrate how real-time sensor occupancy updates the interactive parking grid in the application.

Survey Results

The survey items and the mean scores collected from the 15 participants are detailed in Table 5.4:

ID	Survey Statement	Score (Out of 5.0)
Q1	The mobile application interface is visually appealing, modern, and easy to navigate.	4.80
Q2	Registering a vehicle and generating the QR code is straightforward.	4.53
Q3	Scanning a QR code to notify a blocking vehicle owner is fast and highly intuitive.	4.47
Q4	The real-time interactive grid accurately reflects the parking slot availability.	5.00
Q5	Booking a spot and authorizing payment via Stripe card feels secure and convenient.	5.00
Q6	Configuring 2FA and editing system permissions is simple to execute.	4.80
Q7	The system protects user privacy by eliminating the need to display phone numbers on the vehicle	5.00
Q8	Overall, myMove is a highly effective solution compared to traditional parking systems.	4.87

Table 5.4: User Acceptance Testing (UAT) Survey Results



RESPONSE BREAKDOWN

Rating Option	Count	Percentage
5 - Strongly Agree	99	82.5%
4 - Agree	19	15.8%
3 - Neutral	2	1.7%
2 - Disagree	0	0.0%
1 - Strongly Disagree	0	0.0%
Total	120	100.0%

Figure 5.4: Overall User Acceptance Testing (UAT) Results from Q1 - Q8

5.4 Feedback & Discussion

The feedback gathered from the post-testing survey and interviews was overwhelmingly positive. The thematic analysis of the participants' open-ended responses highlighted several key strengths and areas for improvement:

Key Appreciated Features:

1. **Privacy-Preserving QR Communication:** Participants appreciated the QR feature because it allows drivers to be contacted without sharing personal phone numbers. This helped address privacy concerns when dealing with double-parking situations.
2. **Easy Access for Guests:** Users appreciated that guests can scan the QR code and send notifications without installing the mobile app. This makes the system more convenient and practical for wider use.
3. **Real-Time Parking Updates:** Participants praised the ESP32 prototype for updating parking availability instantly in the mobile app, making it easier to find available spaces.
4. **Convenient Vehicle Management & Payment:** Participants liked being able to register multiple vehicles under one profile. They also found the Stripe payment process smooth, fast, and secure.

Areas for Improvement:

1. **Functional & Security Enhancements:** Adding custom emergency contacts (e.g., family or friends) to the SOS security features, and incorporating a countdown timer on the screen when a parking spot is reserved in the homepage.
2. **Navigation & Booking Convenience:** Integrating navigation systems (Google Maps or Waze) to guide users to their slots, and providing a "Save Favorite Slots" feature for recurring reservations.
3. **Payment Options:** Expanding payment gateways to include local e-wallets (e.g., Touch 'n Go eWallet) and QR-code payments, along with a digital wallet in-app to pre-load parking funds.
4. **User Interface & Visual Options:** Implementing a manual Light Mode toggle switch to accommodate personal preference, alongside a history section for expense tracking and PDF receipt downloads.
5. **Physical & Environmental Considerations:** Transitioning the physical hardware from ultrasonic sensors to camera-based or magnetic loop sensors for better outdoor durability, and providing in-app functionality to report physical obstacles or broken sensors in a slot.
6. **Accessibility:** Introducing a Bahasa Melayu translation option to make the app more inclusive and user-friendly for local drivers.

5.5 Discussion of Practical Implication

The results of the system evaluation and UAT carry significant implications for multiple stakeholders within the smart community framework:

1. The high satisfaction score for QR communication (Q7, Mean: 5.00) shows that myMove helps reduce the stress of double parking by providing a simple and secure way to contact vehicle owners.
2. The fast sensor updates (1.1s) and SOS alerts (0.8s) show that the system provides real-time information and supports quick responses to emergencies.
3. The simple Web-Scanner shows that guest users can access key features without installing the full app.
4. The project provides a practical example of combining ESP32, Firebase, and Stripe to develop a smart parking system.

5.6 System Limitation

While the evaluation proves the myMove system is stable and fully functional, the following technical and operational limitations were identified:

1. **Internet Dependency:** Real-time updates and QR communication require an active internet connection. Poor connectivity may delay updates and notifications.
2. **Language Limitation:** The app currently supports only English, which may limit use among non-English-speaking drivers.
3. **Prototype Limitations:** The ultrasonic sensors were tested only in a tabletop prototype and may be affected by outdoor conditions such as dust, mud, and temperature changes.

5.7 Summary

Chapter 5 presented the complete testing and evaluation results of the myMove: IoT Enabled Smart Parking Reservation and Privacy Vehicle Communication System. Unit testing verified the individual modules with a 97.8% pass rate, while integration testing confirmed that the hardware, backend databases, and mobile app communicate flawlessly with 100% success. System testing showed low latency, with sensor occupancy syncing in 1.1 seconds and double-parking notifications arriving in under 2 seconds. Finally, User Acceptance Testing with 15 participants validated the system's usability and privacy features, achieving high satisfaction ratings. These outcomes demonstrate that the system is fully functional, secure, and ready to address real-world smart parking challenges.

CHAPTER 6

CONCLUSION

6.1 Introduction

This final chapter concludes the thesis by summarizing the myMove: IoT Enabled Smart Parking Reservation and Privacy Vehicle Communication System project. It reviews the core motivations and achievements of the system, provides a detailed evaluation of how each research objective was fulfilled, details the technical and practical contributions of the work, acknowledges existing limitations, and outlines actionable recommendations for future enhancements. Urban parking congestion, inefficient spot searching, privacy risks associated with leaving contact details on windshields, and double-parking disputes continue to create significant friction for daily commuters in Malaysia. The myMove system was conceptualized and developed as an integrated IoT hardware and multi-platform software ecosystem to solve these challenges, delivering real-time occupancy updates, seamless spot reservations, secure card payments, emergency SOS alerts, and privacy-preserving QR-based driver communication.

6.2 Project Summary

The myMove project followed the traditional Waterfall software development methodology, progressing systematically from preliminary literature review and user requirements definition to system design, hardware assembly, full-stack software development, cloud deployment, and rigorous system evaluation.

The complete ecosystem comprises five interconnected core deliverables:

1. **myMove Flutter Mobile Application:** Executed and validated on an Android physical smartphone, providing drivers with real-time interactive parking grid views, spot reservations, integrated Stripe credit card payments, vehicle profile management, Two-Factor Authentication (2FA) security, and an in-app driver communication hub.
2. **myMove Web Admin Dashboard:** Developed with React, TypeScript, and Vite for parking lot operators and security managers. It supports location and slot CRUD operations, live parking occupancy monitoring, revenue analytics, and instant SOS emergency alarm routing.

3. **myMove Scanner-Web Guest Portal:** A lightweight, zero-install mobile web interface accessible by scanning a vehicle's windshield QR code. It enables guest drivers to notify vehicle owners of double-parking blockages anonymously without needing to install the mobile app.
4. **IoT Hardware Tabletop Prototype:** A physical 3D prototype equipped with an ESP32 microcontroller, ultrasonic distance sensors (HC-SR04), status LEDs, and custom C++ firmware that calculates distance changes and streams real-time occupancy events to the cloud.
5. **Serverless Cloud Infrastructure:** Powered by Firebase (Authentication with 2FA TOTP secrets, Cloud Firestore for user/vehicle/booking records, Realtime Database for high-speed sensor streams, and Cloud Functions for Stripe checkout tokenization and FCM push notification dispatches).

Throughout multi-level testing, including unit testing (97.8% pass rate), integration testing (100% pass rate), system performance testing (latencies below 2.0s), and User Acceptance Testing (UAT) with 15 real users (overall score of 4.81 out of 5.00). The myMove platform has demonstrated strong performance, reliability, and usability as a smart parking solution

6.3 Achievement of Project Objectives

The overarching goal of this research was to develop and evaluate a comprehensive smart parking and communication system to reduce driver stress, search time, and double-parking friction. The project successfully achieved all three research objectives defined in Chapter 1:

1. Research Objective 1:

“To study existing parking systems, identify current issues faced by Malaysian drivers, analyze the limitations of current car parking application, define user requirement specification and features through literature review and survey”

- **Status:** Fully Achieved
- **Outcome:** An extensive literature review of existing market solutions (such as JomParking and Flexi Parking) was conducted, highlighting key limitations including lack of real-time spot-level hardware monitoring, privacy exposure from windshield contact numbers, and delayed payment confirmations. User requirements were compiled to establish functional and non-functional specifications, defining the core architecture, data schemas, and UI wireframes presented in Chapter 4.

2. Research Objective 2:

"To develop the myMove application (mobile and web admin) by implementing its core features, including real-time parking search, hardware-integrated spot monitoring, and booking."

- **Status:** Fully Achieved
- **Outcome:** The full myMove ecosystem was engineered and successfully deployed. Key functional implementations include:
 - **IoT Hardware Integration:** ESP32 microcontrollers sync physical spot occupancy to Firebase RTDB in real time (1.1s average latency).
 - **Mobile App & Web Admin:** Drivers can search, reserve, and pay for parking spots using myMove and Stripe, while administrators manage parking facilities and receive SOS alerts via a React web portal.
 - **Privacy-Preserving Contact:** A zero-install Web-Scanner portal dispatches instant Firebase Cloud Messaging (FCM) notifications to vehicle owners upon QR scan without exposing personal phone numbers.

3. Research Objective 3:

"To evaluate the application usability and user satisfaction by conducting User Acceptance Testing (UAT) with real users and collecting feedback for further improvement."

- **Status:** Fully Achieved
- **Outcome:** The system underwent software testing (unit, integration, and system latency checks) followed by UAT involving 15 real participants (students, staff, and daily commuters). The UAT survey yielded an overall average score of 4.81 out of 5.00, with 100% participant agreement on real-time grid accuracy (Q4, Mean: 5.00), payment security (Q5, Mean: 5.00), and privacy protection (Q7, Mean: 5.00). Qualitative feedback confirmed high satisfaction and provided valuable enhancement recommendations.

A structured summary mapping each research objective to its achievements presented in Table 6.1 below:

Research Objective	Status	Key Deliverable	Result
RO1: Study existing systems, identify issues, and define user requirements.	Achieved	Literature review, requirement specification documents, and architecture design.	Identified functional gaps in existing apps; established full PRD specs.
RO2: Develop myMove mobile app, web admin dashboard, and IoT spot monitoring.	Achieved	Flutter app, React Web Admin, Scanner-Web, ESP32 prototype, Firebase backend.	100% feature completion across mobile, web, backend, and hardware integration.
RO3: Evaluate system usability, technical performance, and user satisfaction via UAT.	Achieved	Unit/Integration/System tests, 15-participant UAT survey, and qualitative analysis.	97.8% Unit Pass Rate, 100% Integration Pass Rate, 4.81 / 5.00 UAT Satisfaction.

Table 6.1: Summary of Research Objectives

6.4 Contribution of Systems

The development and evaluation of the myMove system provide valuable practical, operational, technical, and academic contributions:

1. Contribution to Drivers & Commuters:

- **Privacy-Preserving Double-Parking Solution:** Eliminates the risky practice of displaying personal telephone numbers on vehicle windshields, replacing it with a secure QR code scanner flow.
- **Reduced Parking Search Stress:** Interactive real-time slot grids allow drivers to locate and reserve available spots instantly, cutting down fuel consumption and urban traffic congestion.
- **Enhanced In-App Security:** Offers Two-Factor Authentication (TOTP) and an instant SOS emergency button for enhanced personal safety within parking facilities.

2. Contribution to Parking Lot Operators & Facility Management:
 - **Real-Time Operational Visibility:** Provides administrators with live occupancy monitoring and automated visual metrics without requiring manual patrols.
 - **Rapid Emergency Response Routing:** High-priority SOS alerts sound audible alarms and display exact user locations on the admin web dashboard within 0.8 seconds.
 - **Centralized Spot Management:** Flexible web tools allow operators to modify pricing, add parking bays, and view revenue analytics dynamically.
3. Contribution to Software & UX Engineering:
 - **Zero-Install Guest Interaction Pattern:** Demonstrates the effectiveness of using lightweight, web-based endpoints for secondary users (guest scanners), avoiding the friction of forcing full app installations.
 - **Multi-Platform Ecosystem Integration:** Serves as a reference implementation for combining Flutter mobile clients, React web dashboards, and serverless cloud architectures (Firebase).
4. Academic & Technical Contribution:
 - **Low-Cost IoT-Cloud Hardware:** Provides a practical framework for integrating affordable ESP32 microcontrollers with real-time cloud databases (Firebase RTDB) to achieve sub-1.5-second status synchronization in smart city applications.

6.5 System Limitation

While myMove successfully meets all core functional and performance requirements, the following technical and operational limitations were identified during testing:

1. **Dependence on Continuous Internet Connectivity:** Real-time sensor synchronization, Stripe checkouts, and FCM push notifications rely entirely on active internet connections. In deep underground or basement parking facilities with weak cellular coverage, data streams may stall.
2. **Prototype Sensor Hardware Scope:** The physical prototype utilizes HC-SR04 ultrasonic distance sensors, which are cost-effective for tabletop demonstrations but sensitive to rain, dust, and temperature changes in outdoor environments (real-world).

3. **Language Scope:** The mobile application, web dashboard, and notifications are currently available only in English, which may present accessibility barriers for non-English speaking drivers in Malaysia.

6.6 Future Enhancement & Recommendation

Based on user feedback gathered during User Acceptance Testing and technical insights from the development phase, the following enhancements are recommended for future iterations of myMove:

- **Computer Vision & License Plate Recognition (ALPR):** Replace ultrasonic sensors with overhead AI-powered cameras capable of detecting slot occupancy and automatically capturing license plates, reducing maintenance costs and expanding outdoor coverage.
- **Industrial Magnetic & Radar Sensors:** Deploy ground-embedded geomagnetic loop or radar sensors for outdoor parking lots to withstand extreme weather conditions.
- **Augmented Reality (AR) & Indoor Mapping:** Implement AR indoor camera overlays or indoor floor plan maps to guide drivers directly to their specifically reserved parking bay (e.g., "Level 2, Bay B-14").
- **Local Payment Options:** Expand beyond Stripe to integrate popular Malaysian payment gateways (e.g., Touch 'n Go eWallet, FPX Online Banking, and DuitNow QR).
- **Prepaid myMove Wallet:** Allow users to pre-load funds into an in-app digital wallet for instant micro-transactions without re-authorizing credit cards.
- **Multi-Language Toggle:** Introduce Bahasa Melayu and Mandarin Chinese language options across the Flutter app, Web Admin dashboard, and Web-Scanner portal to increase inclusivity for all Malaysian drivers.
- **Countdown Timers & Automatic Slot Release:** Include visible countdown timers for active reservations and automatically release unclaimed slots if the driver does not arrive within a 15-minute grace period.
- **Extended Emergency Contact Alerts:** Allow users to add custom emergency phone numbers (e.g., family or friends) to receive automated SMS alerts when the SOS button is triggered.

6.7 Summary

Chapter 6 presented the conclusion of the thesis report. It summarized the overall project scope and key deliverables, verified the accomplishment of all three research objectives with supporting metrics, detailed the practical and academic contributions of the system, recapped technical limitations, and proposed concrete future recommendations. With the completion of this final chapter, the myMove thesis report is complete, robust, and aligned with academic standards.

REFERENCES

- Zhen C., Z. Xu, Kang T., S. Jia (2025) *Environmental and Social Benefits of Urban Parking Space Shortages Mitigation Management Model: A System Dynamics and Nudge Approach*. 17(14), 6414–6414.
- Group, A., & Group, A. (2024, December 23). *Efficient Parking Management Systems for Smart Cities - Abdul Rahman Al Shareef Group*. Abdul Rahman al Shareef Group.
- Khalid, H. G., Abbas, H. H., Hussein, M. J., Mahmood Jawad Abu-AlShaeer, Mukhlif, S. K., & Dmytro Khlaponin. (2024). *Unveiling the Next Frontier: The Future of Connected Technologies*. 207–218.
- Puspitasari, D., Noprianto, N., Afif Hendrawan, M., & Andrie Asmara, R. (2021). Development of smart parking system using internet of things concept. *Indonesian Journal of Electrical Engineering and Computer Science*, 24(1), 611.
- Kalašová, A., Čulík, K., Poliak, M., & Otahálová, Z. (2021). Smart Parking Applications and Its Efficiency. *Sustainability*, 13(11), 6031.
- Haslinda A., R. Ng, Rosmah M. (2016) *Factors Influencing Haphazard Parking in The Klang Valley, Malaysia* 10.6007/IJARBSS/v6-i9/2303
- Anis.(2025) *KL Sees Over 1,000 Abandoned Cars in Just Five Months*
<https://www.carz.com.my/webview/2025/07/kl-sees-over-1000-abandoned-cars-in-just-five-months>.

Aditya Kumar. (2022, December 20). *Has Technology Improved Our Lives?* | Simplilearn.
Simplilearn.com; Aditya Kumar.

<https://www.simplilearn.com/how-has-technology-improved-our-lives-article>

Najihah Mad Rosni, N., Fadzil Jobli, A., & Negin, M. (2019). Driver Parking Behaviour: An Observational and Experimental Intervention Study. *IOP Conference Series: Materials Science and Engineering*, 601(1), 012002.

Sampath, L. P., & Boggavarapu, K. S. S. (2025). *IoT-based Smart Parking System for Industry 4.0 with QR code access and real-time navigation*. *EPJ Web of Conferences*, 336, 03003.

Siti Aslinda Mahat, & Maslinda Mohd Nadzir. (2025). *ASSESSING USER SATISFACTION OF SMART PARKING PAYMENT SYSTEM IN SELANGOR*. *Journal of Digital System Development*, 3(1), 13–28.

Lim, B. (2017, July 31). *Uni students develop smart system to curb double parking problems*. NST Online; New Straits Times.

<https://www.nst.com.my/lifestyle/bots/2017/07/262578/uni-students-develop-smart-system-curb-double-parking-problems>